The Fabius Function in Lean

A Guided Walkthrough of the `ProveIt` Formalization

Proof architecture, exact computation, asymptotics, and effective approximation

Human-readable guide to Vladimir Reshetnikov's `ProveIt` repository

Repository source snapshot: 25 August 2026

commit `eaea1b9afe2b7ff0b7dd880bd1710e303dec6d80`

Abstract

The Fabius function is a classical example of a smooth function with an elementary-looking self-similarity law and unexpectedly rich arithmetic, Fourier-analytic, probabilistic, and asymptotic structure. Vladimir Reshetnikov's `ProveIt` repository contains a large Lean 4 development that formalizes both of Juan Arias de Reyna's 2017 papers on the function, proves the existence and uniqueness of a canonical bounded solution, constructs its signed global extension, verifies exact evaluation algorithms at dyadic rationals, develops moment and denominator theory, reconstructs Rvachev's Fourier characterization, and goes substantially beyond the papers through q-binomial identities, discrete-limit theorems, sharp shape and inverse theory, computability, non-elementarity results, corrected saddle-point asymptotics, and Legendre expansions.

This document is a proof-oriented walkthrough of that codebase. Its aim is not to restate every declaration in source order. Instead, it explains why the development is split into its present modules, which dependencies are mathematically essential, which are merely packaging dependencies, and how the principal proofs are made acceptable to Lean. Particular attention is paid to the two parallel foundations of the project—exact rational arithmetic and analysis under an abstract `IsFabius` specification—and to the bridge that identifies them. The discussion also records source corrections that become unavoidable in a formal proof, recurring proof-engineering idioms, recommended reading paths, and practical advice for extending the library.

Reader's map. A mathematically oriented reader should begin with Sections 2, 5, 7, 8, 10, and 15. A Lean developer should read Sections 1, 3, 4, 21, and 22. Readers interested in the two source papers can use the theorem map in Appendix B; readers trying to locate a particular implementation should use the annotated module guide in Appendix D; and readers reproducing trust checks should use Appendix H.

Scope and conventions

The repository is active source code, and module boundaries can evolve. Names and dependencies described here refer to commit `eaea1b9afe2b7ff0b7dd880bd1710e303dec6d80` on the `codex/fabius-both-papers` branch, inspected on 25 August 2026. The walkthrough quotes identifiers and very short fragments where useful, but it paraphrases proof bodies rather than reproducing them. The exact Lean source remains the authoritative statement of hypotheses, namespaces, coercions, and imported lemmas. Claims described as questions, conjectures, conditional results, audits, or counterexamples are not silently promoted here to proved positive theorems. The only exceptions are passages explicitly labelled as post-snapshot and the corresponding rows in Appendix D; their individual source checkpoints are recorded with that addendum.

Mathematically, two functions must be kept distinct throughout:

- the bounded cumulative-distribution form $F\colon \mathbb{R} \to [0,1]$, equal to 0 on the left of the unit interval and to 1 on the right; and
- the compactly supported even Rvachev up-function up, obtained by folding or differentiating the bounded form and naturally living near $[-1,1]$.

The code also defines a signed global extension of the bounded function. It is this extension, rather than the clamped bounded map, that satisfies certain translation and dyadic identities for arbitrary real arguments. Several apparent contradictions in informal accounts disappear once these three objects are not conflated.

Lean names are displayed in a monospaced face, for example `Fabius.IsFabius`, `Fabius.fabiusDyadicValue`, and `FabiusSharpAsymptotic.lean`. Mathematical equations are written in conventional notation even when the formal theorem uses a more elaborated type, a subtype coercion, or a filter formulation.

Unless stated otherwise, empty finite sums are 0, empty finite products are 1, and Lean's natural-power convention makes $0^0 = 1$. The Fourier transform convention used in this walkthrough is

$$\widehat{f}(\xi) = \int_{\mathbb{R}} f(x) e^{-2\pi i x \xi}\, \mathrm{d}x.$$

These conventions matter at base indices and in every Poisson- or Fourier-normalization check.

Contents

1 Building and entering the development

1.1 Repository location and toolchain

The relevant source tree is

`Analysis/FabiusFunction/Lean/`

and the umbrella import is `FabiusFunction.lean`. The Lake package declares a Lean library named `FabiusFunction` whose source directory is the path above. At the inspected snapshot, `lean-toolchain` selects `leanprover/lean4:v4.32.0`; `lake-manifest.json` pins a corresponding mathlib revision and the transitive packages used by mathlib, including Aesop, Batteries, Qq, proof widgets, and the import-graph tooling.

The normal build command from the project root is:

```
LAKE_JOBS=1 LEAN_NUM_THREADS=0 lake build +FabiusFunction
```

The source claims in this walkthrough are pinned to commit `eaea1b9afe2b7ff0b7dd880bd1710e303dec6d80`. Before reproducing a declaration or module count, check the checkout with `git rev-parse HEAD`; a moving `main` branch is not a reproducible specification. After dependencies have been installed, a focused edit loop can build one module at a time with a Lake target such as `LAKE_JOBS=1 LEAN_NUM_THREADS=0 lake build +FabiusFunction.DyadicAnalytic`. The root-module build remains the release check.

A useful first test file is:

```
import FabiusFunction

open Fabius

#check IsFabius
#check fabius_spec
#check rvachevUp
#check fabiusDyadicValue
#check extendedFabius
```

Importing the umbrella module is convenient for exploration. During development, importing a narrow leaf module is preferable: it shortens rebuilds, makes accidental dependencies visible, and helps preserve the intended layering.

1.2 What the umbrella module exposes

The top-level `FabiusFunction.lean` does not contain the development itself. It is an import façade. Its principal imports collect six broad products:

1. the formalizations of the two 2017 papers, through `Paper05442.lean` and `Paper06487.lean`;
2. the independent asymptotic project and the k-fold Thue–Morse audit, through `PaperFabiusAsymptotic.lean` and `PaperKFoldThueMorse.lean`;
3. negative-Laplace, Mellin, periodic-correction, and probability-transform interfaces;
4. translated Legendre expansions, least-squares optimality, q-binomial binary reduction, and uniform spline approximation;
5. the sharp shape API—monotonicity, convexity, optimal Lipschitz and derivative bounds, exact analytic loci, and endpoint flatness; and

6. the totalized inverse, non-elementarity and inverse-branch obstructions, Grzegorczyk-style computability, closed saddle jets, and the explicit second saddle correction.

This is a deliberate public surface. The umbrella import answers the question "what has the repository established?" It is not the best answer to "where is this theorem proved?" For the latter, one should descend into the paper aggregators or the subject-specific module families.

1.3 Four kinds of source module

A useful architectural distinction is that a source file usually has one of four roles:

Reusable infrastructure.
Files such as `LegendrePolynomial.lean`, `LegendreSeriesConvergence.lean`, `SaddleAllOrders.lean`, and the Gaussian-polynomial family develop machinery not tied to one canonical Fabius term.

Fabius-specific bridges.
Files such as `DyadicAnalytic.lean`, `AnalyticMoments.lean`, `GlobalExtension.lean`, and `EndpointLaplaceComparison.lean` identify two representations of the same object. These are often the most informative files to read.

Paper-facing façades.
The four `Paper*.lean` aggregators expose source claims in source vocabulary while delegating proofs to stronger internal declarations.

Audit and correction modules.
Files such as `DraftCounterexamples.lean`, `PoissonSummation.lean`, and `FabiusWikipediaObstruction.lean` isolate a false printed claim or a missing hypothesis and expose the checked correction or obstruction.

1.4 Naming conventions that encode intent

Several recurring name patterns help locate the right abstraction:

Generic versus canonical.
A theorem parameterized by `(F:BoundedFabius)(hF:IsFabiusF)` applies to every implementation; a `canonical_` or bare `fabius_` wrapper specializes it to `fabius` and `fabius_spec`.

Exact-to-analytic casts.
Names ending in `_cast` are the load-bearing bridges from rational computation to real analysis.

Convergence forms.
A `HasSum` theorem records a convergence witness; a companion theorem phrased with `tsum` usually provides the convenient equality. Filter and asymptotic APIs use a similar strong-core, wrapper pattern.

Integer scales.
Names containing `_zpow` use an integer exponent, which matters when a dyadic or asymptotic scale can cross 1.

Specifications.
Predicates such as `IsFabius` and `IsOriginalFabius` separate a characterization from a construction and permit independent existence and uniqueness routes.

1.5 A practical exploration workflow

A productive workflow in an editor with the Lean language server is:

1. open the theorem used by a paper aggregator;
2. inspect its type before its proof;
3. use "go to definition" on the first nontrivial helper;
4. note whether the helper belongs to the exact-arithmetic or analytic side of the project;
5. follow imports upward only until the mathematical mechanism becomes clear; and
6. return to the wrapper theorem to see how casts, side conditions, and notation are discharged.

The wrapper theorem is often intentionally short. The real content may be a sequence of modules: one establishing a recurrence over $\mathbb{N}$ or $\mathbb{Q}$, one proving an analytic recurrence for an arbitrary object satisfying `IsFabius`, and one proving that both recurrences share the same initial values.

Architectural point. The development is easier to understand as a directed acyclic graph than as a directory listing. Exact arithmetic starts without the canonical analytic function. Generic analysis starts from an abstract specification rather than from a constructed term. Only after both sides are mature does the code identify exact rational expressions with analytic values and package a canonical function.

---

2 The mathematical objects and their Lean interfaces

2.1 The bounded function

The foundational module `Basic.lean` introduces the closed unit interval as a subtype and uses it as the codomain of a bounded map. Schematically, the first layer has the shape

```
abbrev UnitInterval := Set.Icc (0 : Real) 1
abbrev BoundedFabius := Real -> UnitInterval
```

The subtype codomain records the range bound in the type itself. The projection to $\mathbb{R}$, which the code exposes through a convenience definition such as `fabiusReal`, is used for calculus and integration. This division of labor avoids reproving $0 \le F(x) \le 1$ every time a measure-theoretic bound is required, while still giving the differentiable real-valued function expected by mathlib's analysis APIs.

The abstract predicate `IsFabius` packages the characteristic properties of a candidate. Its fields express, in essence:

$$\begin{aligned} F(x) &= 0 && (x \le 0),\\ F(x) &= 1 && (1 \le x),\\ F(x) + F(1-x) &= 1 && (0 \le x \le 1),\\ F'(x) &= 2F(2x) && (0 \le x \le \tfrac{1}{2}), \end{aligned}$$

together with global smoothness. The precise formal structure uses pointwise derivative predicates and interval membership. Later lemmas extend the symmetry law from the core interval to a statement valid for all real x, with tail cases handled separately.

2.1.1 Why specify before constructing?

Most analytic theorems are stated for a variable F with hypothesis `hF:IsFabiusF`. This is an important architectural choice. It allows derivative identities, Taylor formulas, moment recurrences, and dyadic uniqueness to be proved without unfolding the fixed-point construction. The existence module can then instantiate this generic API, while uniqueness is proved once at the level of the specification.

This organization resembles an interface-and-implementation pattern:

- `Basic.lean` supplies the interface;
- `Differential.lean`, `DyadicAnalytic.lean`, and related modules prove generic consequences of the interface;
- `Existence.lean` constructs `Existence.boundedCandidate` and proves unique existence through `existsUnique_fabius` and `IsFabius.eq`; and
- `PaperStatements.lean` selects the canonical term `fabius` from that witness and exports `fabius_spec`, letting later files forget the implementation again.

2.2 The Rvachev up-function

The compactly supported function up is the symmetric "twin" of the bounded cumulative form. The code defines `rvachevUp` by folding the bounded function around the center of the unit interval. Up to the chosen normalization and translation, the relation is the familiar one

$$\mathrm{up}(x) = F(x+1) \quad (-1 \le x \le 0), \qquad \mathrm{up}(x) = F(1-x) \quad (0 \le x \le 1),$$

with zero outside the support. Symmetry gives the equivalent positive-branch expression $1 - F(x)$ on the unit interval. The exact Lean definition is arranged to make evenness and support proofs tractable and to match the Fourier normalization used in the original paper.

The up-function is the more natural object for Fourier analysis because it is even, smooth, compactly supported, and normalized to have integral one. The bounded form is the more natural object for dyadic arithmetic because its derivative equation and boundary values generate finite Taylor recurrences.

2.3 The signed global extension

The bounded function cannot satisfy an unrestricted translation law: it is constant on both tails. The repository therefore defines `extendedFabius`, a signed global extension assembled as a locally finite sum of odd translates of up, weighted by the Thue–Morse sign. In the repository's normalization,

$$\widetilde{F}(x) = \sum_{n \ge 0} (-1)^{s_2(n)} \ \mathrm{up}(x - 2n - 1).$$

Only finitely many summands are nonzero at any given x. Lean nevertheless treats the expression through a `tsum`, so the library must prove local finiteness, summability, and the reduction of the infinite sum to a finite set before elementary rearrangements are legal.

On the unit interval the extension agrees with the bounded function. Away from that interval it carries the Thue–Morse block signs needed for global dyadic formulas. This distinction is used pervasively in `GlobalExtension.lean`, `GlobalDyadic.lean`, and the q-binomial modules.

Formalization-sensitive point. When an informal statement writes $F(x)$ for all real x, inspect the theorem's target. It may refer to the bounded map, the real projection of that map, the compactly supported up-function, or the signed extension. Several corrected statements in the repository amount precisely to choosing the right one.

2.4 Auxiliary analytic objects

`Basic.lean` also reserves names for the analytic transforms that later modules identify:

- a complex sinc function with a removable value at zero;
- the Fourier transform and infinite sinc product associated with up;
- moment and half-moment generating series;
- real and complex versions of $(\exp z - 1)/z$, again with the singularity removed; and
- integral definitions of moments over the support.

Defining these objects early stabilizes names and normalizations. Proofs of convergence, equality, and differentiability are deferred to modules with the necessary imports. This keeps `Basic.lean` conceptually small even though it determines the vocabulary of the whole project.

3 The dependency architecture

3.1 A layered view

The following table suppresses many helper modules but captures the principal flow. Arrows point from prerequisites to consumers.

Layer	Principal inputs	Flow	Principal consumers
Exact arithmetic	Arithmetic; parity; normalized moments; denominator bounds	$\longrightarrow$	dyadic closed forms and the exact evaluator
Generic analysis	Basic and `IsFabius`; differential, scale, and translation laws	$\longrightarrow$	`DyadicAnalytic`, the exact–analytic bridge
Canonical object	the bridge plus contraction-based existence and uniqueness	$\longrightarrow$	analytic moments, the signed global extension, Fourier analysis, and probability
Advanced layers	exact evaluation, moments, and the global extension	$\longrightarrow$	q-binomial limits, negative-Laplace asymptotics, paper façades, and Legendre theory

Table 1: Conceptual dependency flow. The source contains finer-grained helper modules between nearly every displayed pair.

3.2 Why existence is not the first analytic file

A newcomer might expect `Existence.lean` to precede every theorem about the function. Instead, the code first proves a great deal about an arbitrary `IsFabius` object. This is not circular. The direction is:

1. assume a candidate satisfies the abstract laws;
2. derive its dyadic values and prove that any two candidates agree on a dense set;
3. construct one candidate by a contraction mapping; and
4. invoke the already-proved generic uniqueness theorem.

This order isolates the hard analytic construction from the algebraic uniqueness mechanism. It also means that the dyadic evaluator is not merely a computational afterthought: it participates in the proof that the canonical object is unique.

3.3 Two independent uniqueness arguments

The repository contains two conceptually different uniqueness proofs.

The first is adapted to the bounded specification. The derivative and symmetry equations determine exact values at every dyadic rational. Dyadics are dense, and the candidates are continuous, so agreement on dyadics implies agreement everywhere.

The second is adapted to Rvachev's original compactly supported characterization. The derivative/refinement law is transformed into a Fourier-domain sinc equation. Iteration forces the entire Fourier transform to equal an infinite sinc product. Fourier injectivity on an appropriate smooth rapidly decreasing class then forces the original functions to coincide.

Keeping both arguments is valuable. The dyadic proof is computational and feeds exact arithmetic; the Fourier proof mirrors the historical characterization and explains the infinite product.

3.4 Paper aggregators versus theorem-producing modules

Files named `Paper05442.lean`, `Paper06487.lean`, and `PaperFabiusAsymptotic.lean` are indexes and audit records. They import theorem-producing modules and expose a source-facing collection of results. For example, `PaperStatements.lean` gives declarations whose numbering and prose correspond to the arithmetic paper, but the actual proof of a denominator theorem may be distributed across `NormalizedMoments.lean`, `Parity.lean`, `TwoAdic.lean`, and `DenominatorBound.lean`.

This separation has three advantages:

- source numbering does not dictate internal dependency boundaries;
- corrected hypotheses can be documented at the aggregator without contaminating low-level lemmas; and
- later developments can reuse strong internal theorems rather than wrappers tailored to a paper's notation.

4 The exact-arithmetic foundation

4.1 Why the development begins over $\mathbb{N}$ and $\mathbb{Q}$

The module `Arithmetic.lean` is deliberately independent of the canonical analytic function. It defines the integer and rational sequences that later turn out to be moments, dyadic values, normalizing factors, and denominator bounds. This has a profound practical advantage: exact identities can be proved using induction, finite sums, divisibility, and polynomial normalization without carrying measurability, differentiability, or convergence hypotheses.

The exact side is not just a transcription of closed formulas. It supplies executable definitions. One can evaluate a dyadic rational by normalization and recursion, and Lean's kernel can reduce the resulting rational expression. The same definitions then serve as specifications for correctness theorems.

4.2 Binary weight and the Thue–Morse sign

The smallest combinatorial ingredients are the binary digit sum and its sign:

$$s_2(n) = \text{number of ones in the binary expansion of } n, \qquad \varepsilon(n) = (-1)^{s_2(n)}.$$

In the code these appear under names such as `binaryWeight` and `thueMorseSign`. Elementary identities under doubling and successor are proved early because they drive both the dyadic evaluator and the q-binomial development:

$$s_2(2n) = s_2(n), \qquad s_2(2n+1) = s_2(n) + 1,$$

so that $\varepsilon(2n) = \varepsilon(n)$ and $\varepsilon(2n+1) = -\varepsilon(n)$.

Lean's natural-number division and remainder operations require explicit control of positivity. Proofs that are a one-line observation on binary expansions often become a short cluster of lemmas about `Nat.div`, `Nat.mod`, powers of two, and bit constructors. Establishing this API once prevents the higher-level files from repeatedly descending to bit arithmetic.

4.3 Product normalizations

Several finite products recur in moment denominators. The arithmetic module names them rather than expanding them at every use. Principal examples include variants of

$$\prod_{j=1}^{n}(2^j - 1), \qquad \prod_{j=1}^{n}(2^{2j} - 1), \qquad (2n-1)!!.$$

The source uses names such as `mersenneProduct`, `evenMersenneProduct`, and `oddDoubleFactorial`. Their basic positivity, nonvanishing, splitting, and divisibility properties are the denominator-management toolkit for later proofs.

A formal proof benefits from keeping two versions of a formula:

- a mathematically transparent quotient in $\mathbb{Q}$; and
- a division-free equality in $\mathbb{N}$, obtained after multiplying by a common product.

The quotient is convenient for stating a moment recurrence. The division-free equality is the robust object for induction, parity, and divisibility, because it avoids repeated side goals asserting that denominators are nonzero.

4.4 Moment and half-moment recurrences

The exact sequences `moment` and `halfMoment` are defined recursively as rational numbers. The first models the even moments $\int_{\mathbb{R}} x^{2n}\,\mathrm{up}(x)\,\mathrm{d}x$. The second has normalized base value $d_0 = 1$ and, for $n \geq 0$, models

$$d_{n+1} = (n+1)\int_0^1 t^n\,\mathrm{up}(t)\,\mathrm{d}t.$$

Its connection to the bounded function's inverse-power values is derived afterward rather than built into the definition. Both exact definitions mirror recurrences obtained analytically by integration by parts and scaling.

Alongside them, `momentNumerator` and `halfMomentNumerator` are natural-number recurrences with the common denominators cleared. Their role is easy to underestimate. They provide:

1. a finite, executable representation of exact values;
2. a target for strong-induction proofs of closed forms;
3. a clean setting for parity arguments; and
4. a bridge to reduced rational numerators and denominators.

A recurring proof pattern is to start from the rational recurrence, multiply by the anticipated global normalizing product, distribute that product through a finite sum, and use factorial or Mersenne product splitting to identify every term with an earlier natural numerator. Once the normalizations are designed correctly, the final goal is a polynomial identity in natural numbers and powers of two.

Lean pattern. For exact rational identities, the development usually proves all denominator factors positive or nonzero near the beginning of the proof, rewrites divisions into a common denominator, and invokes `field_simp` only after the relevant nonzero facts are available. The remaining equality is discharged by a combination of finite-sum rewrites, `ring_nf`, and induction hypotheses. Calling `field_simp` too early tends to produce enormous and poorly shaped goals.

4.5 Closed dyadic formulas

The exact values at inverse powers of two are represented by a sequence such as `halfMomentFabiusValue`. The source-facing closed formula `fabiusDyadic` is total on natural numerators and is a finite sum built from Thue–Morse signs, even binomial coefficients, and the exact moment sequence. After casting, it equals the bounded Fabius value at $a/2^n$ when $a \leq 2^n$, and the signed global extension at that argument for arbitrary natural a. This closed formula is distinct from the inverse-power table used by the faster Horner evaluator.

The formalization maintains a distinction between:

- a closed formula suited to mathematical theorems;
- a Horner form suited to evaluation;
- a unit-interval evaluator that clamps and removes highest set bits; and
- a global evaluator that applies triangular reduction on two-unit cells and a Thue–Morse block sign.

This separation makes correctness modular. One first proves that the Horner program computes the finite polynomial, then that recursive bit reduction computes the closed formula, then that the closed formula equals the analytic function.

4.6 The inverse-power table

The function `fabiusInversePowTwoTable` computes a prefix of exact values

$$F(1), F(2^{-1}), F(2^{-2}), \dots, F(2^{-n}),$$

so the array has size $n+1$. A table-based implementation avoids recomputing the same recurrence at every recursive step. Correctness is expressed through prefix stability: extending a table preserves all previously computed entries. This is essential because the evaluator may request values in an order dictated by the bits of the numerator rather than by a single monotone loop.

Formal prefix stability is expressed through array size and optional `Array.getElem?` lookup, together with compatibility of `Array.push` and the defining recurrence. Such lemmas look implementation-specific, but they prevent computational code from leaking into the mathematical correctness theorem.

4.7 Horner evaluation

The local Taylor polynomial is evaluated by `fabiusTaylorHorner`. Horner's rule is not only computationally efficient; it is proof-friendly. The correctness theorem is an induction over the coefficient list, and the induction invariant is a suffix polynomial. In contrast, a direct power-sum implementation forces repeated reasoning about exponents, list indices, and truncated ranges.

The coefficients are rational and involve powers of two coming from iterated differentiation. The proof therefore includes cast-normalization lemmas ensuring that a natural exponent or binomial coefficient has the same interpretation in $\mathbb{N}$, $\mathbb{Q}$, and $\mathbb{R}$. The code frequently uses explicit type annotations at precisely these interfaces; their apparent verbosity documents where coercion inference would otherwise be ambiguous.

4.8 Recursive removal of the highest bit

At the center of the evaluator is a structurally terminating function such as `fabiusDyadicUnitAux`. Given a dyadic numerator a and exponent n, it removes the highest set bit of a. Write

$$a = 2^b + r, \qquad 0 \le r < 2^b.$$

A Taylor identity expresses the value at $a/2^n$ using the value and inverse-power data associated with the smaller remainder r. Since $r < a$ whenever $a > 0$, the recursive call decreases in a well-founded order.

This is the computational counterpart of traversing the ones in the binary expansion of a. The number of recursive Taylor reductions is therefore controlled by $s_2(a)$, not by the magnitude of a itself. Together with precomputation of the inverse-power table, the design yields the approximate complexity described in the repository documentation: a quadratic table phase in the exponent and a bit-weight-dependent evaluation phase.

Lean does not accept "the highest bit was removed" as a termination argument. The source proves a chain of inequalities connecting the bit logarithm, the largest power of two not exceeding a, and the remainder. This chain is packaged so the recursive definition can use a measure. The correctness proof later reuses the very same decomposition, an example of an implementation lemma doing double duty as a mathematical induction lemma.

4.9 Clamping, reflection, and the global evaluator

The unit-interval wrapper `fabiusDyadicUnit` clamps natural numerators at the right endpoint and delegates interior points to highest-bit recursion; it does not perform reflection. The rational-valued `fabiusDyadicValue` accepts an integer numerator and power-of-two denominator, returning 0 for nonpositive inputs and 1 at or beyond the right endpoint. Separate refinement theorems prove that equivalent unreduced representations give the same result.

For the signed extension, `extendedFabiusDyadicValue` reduces the numerator modulo a two-unit grid length, reflects the residue into the unit cell, and applies the Thue–Morse sign of the block index. This is triangular cell reduction, not global periodicity. Multiplying numerator and denominator by a common power of two leaves the output unchanged. Refinement invariance is proved explicitly rather than delegated to quotient types, because the evaluator's input is computational data, not an abstract element of a dyadic localization.

Proof architecture. There are three distinct notions of normalization:

1. arithmetic normalization of a fraction by canceling powers of two;
2. bounded normalization into $[0, 1]$ using the constant tails; and
3. global triangular cell reduction, with a Thue–Morse block sign.

The code proves each normalization preserves the intended mathematical value before composing them. □

4.10 Recognizing dyadic rationals

The definitions `IsDyadicRational` and `dyadicExponent?` support a user-facing exact evaluator for rational inputs. A rational is dyadic when its reduced denominator is a power of two. The decision procedure must connect:

- the internal normalized numerator and positive denominator of $\mathbb{Q}$;
- a Boolean or optional power-of-two test; and
- a proposition asserting the existence of an integer numerator and natural exponent.

The proof that recognition is sound is separate from the proof that the returned exponent and numerator evaluate correctly. This makes failure meaningful: a non-dyadic rational is rejected because no representation with a power-of-two denominator exists, not merely because a search limit was reached.

4.11 Arithmetic API design lessons

The exact core illustrates several design principles that recur throughout the repository.

First, executable functions and theorem-friendly formulas need not be identical definitions. They are connected by named correctness lemmas. Second, normalization data should be made explicit, especially when a mathematical quotient hides representation choices. Third, the strongest useful invariant is often not the final equality but a stability statement about prefixes, refinements, or common denominators. Finally, natural-number recurrences are frequently the right interface for arithmetic theorems even when the motivating quantities are rational.

5 Constructing the canonical bounded function

5.1 The ambient complete metric space

The construction in `Existence.lean` uses Banach's fixed-point theorem. Let

$$I = [0,1], \qquad C = C(I, \mathbb{R})$$

with the uniform metric. Since the codomain is complete and I is compact, the continuous-map space is complete. The code then cuts out a closed subset of C consisting of functions satisfying two elementary conditions:

$$\begin{aligned} 0 \le f(x) &\le 1, \\ f(x) + f(1-x) &= 1. \end{aligned}$$

The source calls the reflection operator and the admissibility predicate by local names and proves that the admissible set is closed. A linear function, essentially $x \mapsto x$, witnesses nonemptiness.

Using a closed subspace rather than the entire continuous-function space is decisive. Symmetry is not recovered after the fixed point; it is an invariant of the contraction. This makes the integral normalization automatic and improves the contraction constant from the naive value one to one half.

5.2 Extending an interval function

An element of $C(I, \mathbb{R})$ accepts a subtype argument. To integrate it over an ordinary real interval, the code defines an extension by projecting an arbitrary real number to I. This projection clamps values below zero to zero and values above one to one. Continuity follows from continuity of the metric projection to a closed convex interval.

For an admissible f, define the cumulative integral

$$A_f(y) = \int_0^y \widehat{f}(t)\ \mathrm{d}t,$$

where $\widehat{f}$ is the clamped extension. Symmetry implies

$$\int_0^1 f(t)\ \mathrm{d}t = \frac{1}{2}.$$

The Lean proof reflects the integral by the substitution $t \mapsto 1-t$, adds the two copies, and uses the pointwise symmetry equation. The subtype coercions and interval maps are handled in helper lemmas so the final normalization proof resembles the paper calculation.

5.3 The nonlinear-looking transform that is actually affine

The fixed-point transform is defined piecewise at the midpoint:

$$(Tf)(x) = \begin{cases} A_f(2x), & 0 \le x \le \frac{1}{2}, \\ 1 - A_f(2-2x), & \frac{1}{2} \le x \le 1. \end{cases}$$

The two branches agree at $x = 1/2$ because $A_f(1) = 1/2$. Each branch is continuous, and the gluing lemma supplies continuity of Tf on the whole interval. Bounds follow because f is bounded between zero and one; symmetry follows by exchanging the two branches.

The transform is affine on the admissible set. The constants in the right branch cancel when two transformed functions are subtracted. This is why the contraction estimate can be proved through integrals of $f - g$.

5.4 The contraction estimate

Let $h = f - g$ for two admissible functions. Symmetry gives

$$\int_0^1 h(t)\ \mathrm{d}t = 0.$$

For $0 \le y \le 1$, the prefix integral may be estimated in two ways:

$$\left|\int_0^y h\right| \le y\|h\|_\infty, \qquad \left|\int_0^y h\right| = \left|\int_y^1 h\right| \le (1-y)\|h\|_\infty.$$

Therefore

$$\left|\int_0^y h\right| \le \min(y, 1-y)\|h\|_\infty \le \frac{1}{2}\|h\|_\infty.$$

This is the key insight of the construction. A crude estimate would give constant one and no contraction. The zero-total-integral identity lets the proof select the shorter of the prefix and complementary suffix.

Lean pattern. The formal estimate is assembled from interval-integral norm bounds rather than from an informal "length times supremum" principle. The code proves integrability of the extended difference, obtains a pointwise uniform bound from the distance in the continuous-map space, applies the interval integral inequality on both subintervals, and rewrites the total integral as zero. A final case split on $y \le 1/2$ selects the useful bound.

Banach's theorem now yields a unique fixed point in the admissible subspace. The source extracts its underlying continuous map and extends it to a bounded real function with constant tails.

5.5 From an integral fixed point to the differential law

A fixed point satisfies an integral equation. On the left half,

$$F(x) = \int_0^{2x} F(t)\ \mathrm{d}t.$$

For the probability CDF this identity is actually valid for every $x \le 1/2$, not only for $x \in [0, 1/2]$: below zero both the CDF and the oriented integral vanish. Lean exposes this stronger form as `weightedSumCDF_eq_intervalIntegral_of_le_half` and retains `weightedSumCDF_left_` `formula` as the interval-shaped compatibility wrapper. The fundamental theorem of calculus and the chain rule give

$$F'(x) = 2F(2x).$$

On the right half, differentiation of the reflected branch yields the corresponding equation through symmetry. The proof handles the midpoint and endpoints separately because standard derivative rules apply most directly on open neighborhoods, while the desired structure requests derivatives at boundary points as well.

The source uses `HasDerivAt` statements whenever possible. Such statements compose cleanly through affine maps and integral primitives; a theorem about `deriv` alone would require differentiability side conditions each time it is rewritten. Endpoint derivatives are obtained by showing that the formulas from adjacent pieces agree, using the tail values and symmetry.

5.6 Bootstrapping smoothness

The fixed-point theorem initially gives only continuity. The integral equation gives one derivative. The derivative is a scaled composition of the original continuous function, so it is continuous. Iterating this observation upgrades the function to every finite differentiability order.

The formal proof is organized around reusable scale-and-derivative lemmas rather than a single enormous induction on `ContDiff`. At stage $n+1$, the derivative formula expresses the new derivative in terms of a scaled version of the n-th derivative. Boundary compatibility follows from vanishing on the tails and symmetry. The final theorem packages all finite stages as C^∞.

5.7 Uniqueness by dyadic density

The contraction theorem already gives uniqueness inside the admissible continuous space. The exported specification, however, is phrased for real functions satisfying `IsFabius`; it is useful to prove uniqueness at this abstract level. The later dyadic analysis shows that every such function has the same exact value at every dyadic point. Since dyadics are dense in $\mathbb{R}$ and both candidates are continuous, the functions coincide.

The proof has the standard topological shape: approximate an arbitrary x by a sequence of dyadics, rewrite equality at each sequence term, and pass to the limit by continuity. The nontrivial work lies earlier, in proving exact dyadic equality without assuming the fixed-point implementation.

Architectural point. The fixed-point construction proves that the interface is inhabited. The dyadic theory proves that the interface is subsingleton. This yields a canonical object whose public properties are independent of the construction details.

---

6 Differential identities and the signed global extension

6.1 The role of `Differential.lean`

`Differential.lean` is the calculus engine for an arbitrary `IsFabius` candidate. Starting from the first derivative law on the left half and the globalized symmetry law, it derives corresponding formulas on other intervals and iterates the derivative relation.

The characteristic coefficient after n differentiations is a triangular power of two. In a typical region where all scaled arguments remain in the appropriate branch, one obtains

$$F^{(n)}(x) = 2^{1+2+\cdots+n}F(2^n x) = 2^{n(n+1)/2}F(2^n x).$$

The exact theorem includes domain conditions ensuring that each intermediate scaled point lies in the branch where the derivative law is valid. Reflection introduces signs and translated arguments on the other half.

These iterated formulas explain three later phenomena at once:

- finite Taylor formulas at dyadic points;
- flatness at the support boundary; and
- extremely rapid decay near zero.

6.2 Domain bookkeeping

An informal derivation often differentiates the relation $F'(x) = 2F(2x)$ without restating its domain. Lean forces the domain to be propagated. If a theorem is iterated n times, the proof must establish that $2^j x$ belongs to the relevant interval for every $j < n$. The source packages this as inequalities involving powers of two and dyadic cells.

This bookkeeping is not bureaucratic. One of the asymptotic draft corrections later in the repository arises because a transformed delay-differential identity is valid only beyond a threshold. The formal derivative theorem makes the threshold impossible to omit.

6.3 Scale and translation lemmas

`ScaleTranslation.lean` collects chain-rule identities for functions of the form

$$x \longmapsto c\,F(ax+b), \qquad x \longmapsto c\ \mathrm{up}(ax+b).$$

These are used by approximation arguments, global extension formulas, Fourier transforms, and splines. The value of a dedicated module is that every later proof can invoke a theorem with all factors of a, signs, and differentiability hypotheses already correct.

A common Lean mistake is to rewrite an iterated derivative of a composition as though the affine map contributed only one factor of a. The library's lemmas state the full a^n factor. Specialized powers-of-two corollaries then combine this with the triangular coefficient from the Fabius differential law.

6.4 Local finiteness of the translate sum

`GlobalExtension.lean` proves that the signed translate series defining $\widetilde{F}$ is harmless. Because up has compact support, only finitely many natural indices n can satisfy $x - 2n - 1 \in [-1, 1]$ for fixed x. More strongly, on a compact neighborhood of x, a single finite set of odd translates contains every nonzero summand.

That stronger local statement is what permits termwise continuity and differentiation. The proof turns support inequalities into natural-number bounds and proves every index outside a finite range contributes zero. The `tsum` can then be rewritten as a finite sum using a summability-by-finite-support lemma.

6.5 Agreement and global identities

Once local finiteness is available, the signed extension is shown to agree with the bounded function on $[0, 1]$. On every cell $2b \le x \le 2b + 2$, exactly one translate survives and

$$\widetilde{F}(x) = (-1)^{s_2(b)}\ \mathrm{up}(x - 2b - 1).$$

The case $b = 0$, together with the defining fold relation, gives unit-interval agreement.

The extension is therefore a sequence of two-unit cells, not a globally periodic function: the sign of cell b is the Thue–Morse value $(-1)^{s_2(b)}$. Scale-and-translation theorems record the precise identities on their valid blocks. `GlobalDyadic.lean` combines this cell structure with exact values to remove assumptions that a numerator be reduced or lie between zero and its denominator.

6.6 Global differential calculus

Local finiteness also permits termwise differentiation and even–odd reindexing. For every $x \in \mathbb{R}$, the theorems `extendedFabius_hasDerivAt` and `deriv_extendedFabius` give

$$\widetilde{F}'(x) = 2\widetilde{F}(2x).$$

Iterating this identity yields `iteratedDeriv_extendedFabius`: for every $k \in \mathbb{N}$ and $x \in \mathbb{R}$,

$$\widetilde{F}^{(k)}(x) = 2^{\binom{k+1}{2}}\widetilde{F}(2^k x).$$

Unlike the bounded derivative formula above, these identities require no branch-domain hypothesis.

6.7 Exact Taylor reduction

On its initial block, `extendedFabius_add_pow_two` says that

$$\widetilde{F}(x + 2^r) = -\widetilde{F}(x) \qquad (r \geq 1,\ 0 \leq x \leq 2^r).$$

A derivative version extends this sign change to an integer scale s whenever the derivative order m is high enough that $0 \leq s + m$. Under that condition, `taylorRemainder_translate` packages the exact cancellation of the two adjacent integral Taylor remainders. Write $R_m(y)$ for `fabiusReductionSummy`. If $n \in \mathbb{Z}$, $x > 0$,

$$2^{-n} \leq x < 2^{-n+1}, \qquad y = x - 2^{-n},$$

then `extendedFabius_reduction`, exported as `proposition_ten`, states

$$\widetilde{F}(x) = -\widetilde{F}(y) + \begin{cases} R_{\mathrm{toNat}(n)}(y), & n \geq 0, \\ 0, & n < 0. \end{cases}$$

Thus the finite correction for $n \geq 0$ is an exact Taylor polynomial, not a truncated approximation with an estimated remainder; for $n < 0$, the formula reduces directly to the power-of-two sign-translation law.

6.8 Why use a `tsum` for a locally finite expression?

A hand proof would select the unique active odd translate on each nonnegative two-unit cell. The repository instead uses a conceptually uniform sum over all natural indices. This pays off in translation and differentiation arguments: the same summand family can be manipulated with standard infinite-sum lemmas. The initial cost is a local-finiteness API; the later algebra becomes much cleaner.

7 Sharp shape, inverse branches, and effective computation

This part of the public API gathers consequences that were once scattered among paper wrappers into a reusable shape theory, then uses that theory to construct the inverse and a certified computable evaluator.

7.1 One global derivative equation

The key simplification is the identity

$$F'(x) = 2\ \mathrm{up}(2x-1) \qquad (x \in \mathbb{R}),$$

proved for every `IsFabius` candidate. Since $\mathrm{up} \geq 0$, the bounded function is globally monotone. The strict theory in `Monotonicity.lean` proves that F is strictly increasing on $[0,1]$ and maps that interval bijectively to itself. On the folded side, up is positive exactly on $(-1,1)$, increases strictly to its unique maximum 1 at 0, and then decreases strictly.

`Convexity.lean` feeds this unimodality back through the derivative equation. The derivative of F increases up to $x = 1/2$ and decreases afterward, with exact range $[0,2]$ and unique maximum 2 at the midpoint. Consequently F is convex on the left half-line and concave on the right, strictly so on the two open halves of its transition interval. The same module now identifies the curvature pointwise:

$$F''(x) = 8\big(\mathrm{up}(4x-1) - \mathrm{up}(4x-3)\big).$$

The declarations `deriv_deriv_fabiusReal_pos_iff`, `deriv_deriv_fabiusReal_neg_iff`, and `deriv_deriv_fabiusReal_eq_zero_iff` say respectively that this is positive exactly on $(0,1/2)$, negative exactly on $(1/2,1)$, and zero off $(0,1)$ or at the midpoint.

7.2 Sharp regularity and flatness

`Regularity.lean` proves that 2 is not merely a convenient Lipschitz constant: it is the least global Lipschitz constant for both F and up. The scaling recurrence then yields, for every $k \geq 0$,

$$\big|F^{(k)}(x)\big| \leq 2^{\binom{k+1}{2}},$$

with equality attained at $x = 2^{-k}$ for positive order. The signed extension and the up-function have parallel sharp bounds in `GlobalBounds.lean`; `BoundedDerivatives.lean` transfers them to the clamped function without confusing its constant right tail with the signed extension.

The effective endpoint bounds are stronger than the qualitative assertion that every positive derivative vanishes. If $x \geq 0$ and $2^n x \leq 1$, `SharpFlatness.lean` proves

$$F(x) \leq \frac{2^{\binom{n+1}{2}}}{n!}\, x^n.$$

Reflection gives the analogous estimate for $1 - F(x)$ at the right endpoint. The scale condition is part of the theorem; the family is not a single bound uniform in n.

7.3 The totalized inverse

Strict monotonicity packages the restriction of F to $[0,1]$ as the order isomorphism `fabiusOrderIso`. The definition `fabiusInv` uses the inverse order isomorphism on that interval and `IccExtend` to clamp all real inputs: it is 0 on the left tail and 1 on the right. The module `FabiusInverse.lean` proves both inverse identities on the unit interval, global continuity and monotonicity, endpoint reflection, and exact inversion of every value produced by the bounded dyadic evaluator.

The same module now packages the first-order interior calculus and the exact global smoothness locus. On $0 < y < 1$, `deriv_fabiusInv_eq_inv_two_mul_rvachevUp`, together with `deriv_fabiusInv_pos`, gives

$$(F^{-1})'(y) = \frac{1}{F'(F^{-1}(y))} = \frac{1}{2\,\mathrm{up}(2F^{-1}(y)-1)} > 0,$$

and `fabiusInv_contDiffOn_Ioo` proves C^∞ smoothness on the open unit interval. Because the totalization is locally constant on both open tails, `fabiusInv_contDiffAt_infty_iff` sharpens this to an exact global statement: the inverse is C^∞ at y if and only if $y \neq 0$ and $y \neq 1$. Likewise, `fabiusInv_differentiableAt_iff` says that it is differentiable exactly away from the two clamping endpoints, with `fabiusInv_not_differentiableAt_zero` and `fabiusInv_not_differentiableAt_one` providing the endpoint obstructions. More generally, `fabiusInv_contDiffAt_iff` classifies every order bounded by C^∞: order zero, namely continuity, holds globally, whereas every positive finite order and C^∞ hold exactly off $\{0,1\}$. Its order hypothesis deliberately excludes the stronger analytic order. The divergent one-sided difference quotients quantify the endpoint failure.

The next derivative is also packaged exactly. The declarations `deriv_fabiusInv_hasDerivAt` and `deriv_deriv_fabiusInv` prove

$$(F^{-1})''(y) = -\frac{F''(F^{-1}(y))}{F'(F^{-1}(y))^3} \qquad (0 < y < 1).$$

At the midpoint the exact jet is

$$F^{-1}(1/2) = 1/2, \qquad (F^{-1})'(1/2) = 1/2, \qquad (F^{-1})''(1/2) = 0,$$

by `fabiusInv_half`, `deriv_fabiusInv_half`, and `deriv_deriv_fabiusInv_half`. On $0 < y < 1$, the declarations `deriv_deriv_fabiusInv_neg_iff`, `deriv_deriv_fabiusInv_pos_iff`, and `deriv_deriv_fabiusInv_eq_zero_iff` identify the negative, positive, and zero loci as $(0,1/2)$, $(1/2,1)$, and $\{1/2\}$. The reciprocal reverses the monotonicity of F'. Consequently `strictConcaveOn_fabiusInv_firstHalf` proves strict concavity on the closed interval $[0,1/2]$, while `strictConvexOn_fabiusInv_secondHalf` proves strict convexity on $[1/2,1]$. These closed-interval shape theorems use differentiability only on the interiors, so they are compatible with the singular endpoint slopes.

Flatness of F becomes steepness of F^{-1}. Substituting $x = F^{-1}(y)$ into the effective estimate gives, on the stated shrinking window,

$$y \le 2^{\binom{n+1}{2}} \big(F^{-1}(y)\big)^n,$$

with a factorial-strengthened version as well. In particular $F^{-1}(y)/y \to +\infty$ as $y \downarrow 0$, and reflection supplies the right-endpoint counterpart. More sharply, the positive interior derivative itself tends to $+\infty$ at both ends, by `tendsto_deriv_fabiusInv_atTop_at_zero_right` and `tendsto_deriv_fabiusInv_atTop_at_one_left`. These are one-sided limits through $(0,1)$, not derivative values at the clamping points.

7.4 Exact analytic loci and non-elementarity

The smooth function is analytic precisely off its closed transition interval:

$$\mathrm{AnalyticAt}(F, x) \quad \Longleftrightarrow \quad x \notin [0,1].$$

`NowhereAnalytic.lean` includes both endpoints; outside the interval the function is locally constant. `InverseNotElementary.lean` proves the same analytic locus for the totalized inverse. In the interior, analyticity of the inverse would make F analytic by the inverse-function theorem; at the endpoints, the identity theorem conflicts with the constant tails and strict increase.

The generic modules `AlgebraicBranch.lean` and `ElementaryFunction.lean` show that the formal class `IsElementary` has dense analytic locus, as do continuous algebraic branches with elementary coefficients. `NotElementary.lean` therefore excludes agreement with F on any subset of $[0,1]$ having nonempty interior. `InverseBranch.lean` proves that a continuous right-inverse branch of a densely analytic function is analytically dense on its branch domain. The constructor `IsElementaryOrInverse.invBranch` closes the class under such branches on an open set U only when the chosen total extension is analytic on $\mathrm{interior}(U^c)$; branch-domain boundaries are deliberately excluded. Its Lambert-W theorem is a conditional criterion for any branch satisfying these hypotheses, not an instantiated mathlib declaration of W. `InverseNotElementary.lean` then excludes both F and its totalized inverse from this enlarged class on $(0, 1)$.

Formalization-sensitive point. These are theorems about the explicitly defined Lean predicates `IsElementary` and `IsElementaryOrInverse`, not a claim about every possible computer-algebra language or every informal use of the word "closed form." The precise constructors and branch hypotheses in `ElementaryFunction.lean` and `InverseBranch.lean` are part of the statement.

7.5 Primitive-recursive splines and computable reals

The approximation branch also became effective. `FabiusComputability.lean` proves the uniform estimates

$$\sup_{x\in[0,1]} |S_p(x) - F(x)| \le 2^{-p}, \qquad \sup_{x\in\mathbb{R}} |S_p(x) - \widetilde{F}(x)| \le 2^{-p},$$

and records the primitive-recursive modulus $d(n) = 2n$ coming from the sharp Lipschitz bound. `FabiusComputableSpline.lean` implements the centered Thue–Morse spline using signed pairs of natural numerators, proves its pieces primitive recursive, rounds to a dyadic name with an explicit error budget, and supplies two evaluators with the same error $5 \cdot 2^{-(p+3)}$. The first clamps its input to the bounded interval and packages `IsComputableRealFunction(fabiusRealF)`. The second retains the unrestricted signed spline code; `extendedFabiusSplineApproxPR_error` proves its global error, and `extendedFabius_isComputableRealFunction` packages the result for the signed extension of every F satisfying `IsFabiusF`. Its canonical specialization is `globalFabius_isComputableRealFunction`. Thus both bounded and signed-global computability theorems contain the two required clauses: preservation of computable real sequences and effective uniform continuity. The unrestricted evaluator is a primitive-recursive certificate rather than a complexity claim, since its finite fold length depends on the positive input numerator.

Uniform convergence also controls changing inputs. The theorem `abs_fabiusUniformSpline_sub_extendedFabius_le_add` states

$$|S_p(x) - \widetilde{F}(y)| \le 2^{-p} + 2|x - y|,$$

and `fabiusUniformSpline_tendsto_extendedFabius_of_tendsto` concludes that $S_p(x_p) \to \widetilde{F}(x)$ whenever $x_p \to x$.

8 The exact–analytic dyadic bridge

8.1 The purpose of `DyadicAnalytic.lean`

The exact evaluator is useful only after it is tied to the analytic function. `DyadicAnalytic.lean` is the central bridge. It proves, for every `IsFabius` candidate, the same finite recurrences that were defined independently in `Arithmetic.lean`; then it identifies the two by induction.

The proof has four main stages:

1. derive a finite Taylor identity on a dyadic cell;
2. specialize it to inverse powers of two and obtain an endpoint recurrence;
3. prove that the analytic inverse-power sequence equals the exact rational table; and

4. lift this equality to arbitrary dyadic numerators through the highest-bit recursion.

The surrounding modules `DyadicClosedForm.lean` and `DyadicCorrectness.lean` isolate the algebraic and algorithmic portions of this chain.

8.2 Finite Taylor expansion on a dyadic scale

Fix a sufficiently small dyadic base point. The iterated derivative formula expresses every derivative up to a chosen order through a scaled value of F. At the order where the scaled argument leaves the active interval and enters the zero tail, higher derivatives vanish. Taylor's theorem therefore terminates exactly rather than producing an infinite series with a remainder.

A representative identity, in the normalization used by the evaluator, is

$$F(2^{-m} + y) + F(y) = \sum_{k=0}^{m} \frac{2^{\binom{k+1}{2}}}{k!} F(2^{-(m-k)})\, y^k,$$

valid for $m \geq 1$ and $0 \leq y \leq 2^{-m}$. Thus the highest-bit evaluator computes the displayed polynomial and subtracts the recursively evaluated $F(y)$. The domain and the subtraction are essential parts of the recurrence.

The proof does not invoke analyticity. It uses a finite-order Taylor theorem with a remainder, then proves the next derivative vanishes on the relevant segment. This is crucial: the Fabius function is smooth but nowhere analytic in the appropriate sense, so an argument based on convergence of an infinite Taylor series would be false.

Formalization-sensitive point. Finite Taylor identities are local consequences of compact support and the differential equation. They do not imply that the function equals its infinite Taylor series in a neighborhood. The formalization later turns the same exact truncation phenomenon into a proof of nonanalyticity.

8.3 Inverse-power values

For a candidate F, define informally

$$A_n(F) = F(2^{-n}).$$

The dyadic Taylor identity at a suitable endpoint gives a recurrence for $A_{n+1}(F)$ in terms of earlier $A_j(F)$. Boundary compatibility and symmetry supply the initial value $A_1(F) = 1/2$.

Independently, `Arithmetic.lean` defines a rational sequence satisfying the same recurrence. Strong induction proves

$$A_n(F) = A_n^{\text{exact}}$$

for every n. The induction step is predominantly algebraic, but the statement lives in $\mathbb{R}$; the proof must cast exact rational sums and products into reals. The source localizes these casts in lemmas asserting that the rational recurrence is preserved by the canonical embedding $\mathbb{Q} \hookrightarrow \mathbb{R}$.

8.4 From closed polynomial to Horner program

Once the analytic value is a finite polynomial in exact inverse-power values, `DyadicAnalytic.lean` invokes the algebraic theorem that the closed polynomial equals `fabiusTaylorHorner`. This is a clean interface point:

- analysis proves the value is the mathematical polynomial;
- arithmetic proves the polynomial has exact rational coefficients; and

- algorithmics proves Horner evaluation computes that polynomial.

No proof about derivatives needs to know how a coefficient list is stored, and no proof about lists needs to know the fundamental theorem of calculus.

8.5 Correctness of highest-bit reduction

Suppose the numerator decomposes as $a = 2^b + r$. The analytic Taylor theorem proves exactly the recurrence used by the recursive evaluator. The recursive call has numerator $r < a$, so strong induction on a applies. At the end, the theorem states that evaluating the exact program and embedding its rational output into $\mathbb{R}$ gives $F(a/2^n)$.

The proof handles endpoint and out-of-range cases before entering the recursion. This matches the program structure and avoids forcing the induction hypothesis to cover impossible branch conditions. Each branch ends with a theorem about the corresponding normalization operation: left-tail clamping, right-tail clamping, highest-bit Taylor reduction, or refinement. Reflection appears only in the separate signed global evaluator.

8.6 Representation independence

The same dyadic rational has infinitely many representations:

$$\frac{a}{2^n} = \frac{2a}{2^{n+1}} = \frac{4a}{2^{n+2}} = \cdots .$$

The evaluator's mathematical specification therefore includes refinement invariance. `DyadicCorrectness.lean` proves that one common factor of two can be introduced or removed without changing the result; induction yields invariance under any common power.

There are two possible proof routes. One can prove both representations equal the analytic function and conclude by transitivity, or prove refinement directly from the exact recursion. The repository contains strong exact correctness results so that the executable layer remains trustworthy even when used independently of analysis. Analytic equality then gives a second conceptual explanation.

8.7 Dyadic density and uniqueness

Let F and G satisfy `IsFabius`. Correctness gives

$$F(q) = G(q)$$

for every dyadic rational q. To extend this to an arbitrary real x, choose dyadic approximants such as

$$q_n = 2^{-n} \lfloor 2^n x \rfloor .$$

Then $q_n \to x$. Continuity gives

$$F(x) = \lim_n F(q_n) = \lim_n G(q_n) = G(x).$$

In Lean, density can be applied through a generic theorem saying continuous maps equal on a dense set are equal, or through an explicit approximating sequence. The repository's dyadic infrastructure supplies the density statement in the exact form required by the function-extensionality proof. Tail values allow the argument to focus on $[0, 1]$ when convenient.

8.8 The global dyadic theorem

`GlobalDyadic.lean` transfers exact evaluation from the bounded interval to `extendedFabius`. The result accepts unrestricted integer numerators and nonreduced dyadic representations. It combines:

1. arithmetic decomposition into an integer translate and a unit-interval remainder;
2. the translation law for the signed extension;
3. exact unit-interval correctness; and
4. representation invariance.

This stronger formulation is the one required by the q-binomial identities, where summation indices naturally produce numerators outside $[0, 2^n]$.

Architectural point. The dyadic bridge is the repository's main "commuting square." One path starts with an analytic function and derives a finite Taylor recurrence; the other starts with exact rational recurrences and an evaluator. The correctness theorem says both paths produce the same real number.

9 Moments, generating functions, and exact denominators

9.1 Analytic moments of the up-function

`AnalyticMoments.lean` develops integral interpretations of the exact moment sequences. Because up is smooth and compactly supported, every polynomial moment exists:

$$M_n = \int_{\mathbb{R}} x^n \,\mathrm{up}(x)\, \mathrm{d}x.$$

Evenness immediately gives $M_{2k+1} = 0$, while the exact rational sequence is indexed only by the even moments:

$$M_{2k} = (\mathrm{moment}(k) : \mathbb{R}).$$

The total mass $M_0 = 1$ is proved from the bounded differential equation and the fundamental theorem of calculus, rather than assumed as part of the definition.

The main recurrence is obtained by integration by parts and a dyadic change of variables. The precise coefficients depend on the chosen centering of up, but the structure is a finite convolution of lower moments with powers of two and binomial coefficients. The same recurrence was used to define `moment` in the exact layer.

9.1.1 Integration over compact support

Mathlib's integral on $\mathbb{R}$ is convenient for Fourier transforms, while interval integrals are convenient for integration by parts. The proof repeatedly converts between them using support lemmas. A standard sequence is:

1. prove the integrand vanishes outside a compact interval;
2. rewrite the whole-line integral as an integral over that interval;
3. apply an interval integration-by-parts theorem;
4. show endpoint terms vanish by support or flatness; and
5. transform the remaining integral by an affine substitution.

The support theorem is therefore not merely qualitative metadata; it is an active rewrite rule used throughout the analytic stack.

9.2 Identifying analytic and exact moments

After deriving the recurrence, the module proves by strong induction that

$$M_{2n} = (\mathrm{moment}(n) : \mathbb{R})$$

for every n. Raw odd moments are handled separately by evenness. The displayed formula intentionally suppresses namespace syntax; the Lean theorem states the rational cast explicitly.

The base case is total mass. In the induction step, the analytic recurrence is rewritten term by term using the induction hypotheses. A cast lemma turns the finite rational sum into the corresponding real sum. The exact recurrence then closes the goal.

The same method identifies the normalized integrals of up over $[0, 1]$ with `halfMoment`. The source-facing theorem starts at positive index, while `halfMoment_eq_integral_formula_all` packages the normalized zeroth value and the integral cases into one statement for every natural index. These moments are connected more directly to inverse-power Fabius values, so they become the analytic bridge between moment theory and the dyadic table.

9.3 Generating functions

The analytic moment generating function is represented both as an integral and as a convergent series. Compact support gives uniform bounds

$$|M_n| \le CR^n,$$

which are more than sufficient for convergence of

$$\sum_{n\ge 0} M_n \frac{z^n}{n!}$$

for every complex z. Dominated convergence justifies interchanging sum and integral, giving the usual identity with $\int e^{zx}\,\mathrm{up}(x)\,\mathrm{d}x$.

`MomentPowerSeries.lean` is a separate exact-algebra layer over `PowerSeries` over $\mathbb{Q}$. If

$$M(X) = \sum_{n\ge 0} \frac{\mathrm{moment}(n)}{(2n)!} X^n, \qquad S(X) = \sum_{n\ge 0} \frac{X^n}{(2n+1)!},$$

then its central identity is

$$M(4X) = M(X)S(X).$$

This is a coefficient theorem in exact rational power series, not an appeal to `FormalMultilinearSeries` or `HasFPowerSeriesAt`. Analytic modules later identify the evaluated series with transforms after proving convergence. The half-moment bridge also gives the exact inverse-power formula

$$F(2^{-n}) = \frac{\mathrm{halfMoment}(n)}{n!\,2^{\binom{n}{2}}}.$$

9.4 Removable singularities

Expressions such as

$$\frac{e^z - 1}{z}$$

appear in half-moment generating functions. The source defines a total function with value one at zero, then proves continuity and analyticity across the removable singularity. This avoids carrying $z \ne 0$ through every downstream identity.

The proof combines the exponential power series with division by z term by term. In Lean, defining the value at zero first and proving a global power-series expansion is often cleaner than starting with a piecewise quotient and repeatedly rewriting the nonzero branch.

9.5 Normalized half moments

`NormalizedMoments.lean` proves a closed denominator normalization of the form

$$\mathrm{halfMoment}(n) = \frac{H_n}{(n+1)!\prod_{j=1}^{n}(2^j-1)},$$

where H_n is the division-free natural numerator defined in `Arithmetic.lean`. The theorem is not obtained by asking Lean to normalize a large rational expression automatically. It is proved by strong induction with a deliberately chosen common denominator.

At the induction step, every earlier half moment has a denominator whose factorial and Mersenne products are prefixes of the target denominator. The missing suffix factors are inserted explicitly. Product-splitting lemmas show that multiplying an earlier term by its suffix gives the target common denominator. The natural numerator recurrence was designed to be exactly the resulting cleared equation.

9.6 Normalized even moments

`NormalizedEvenMoments.lean` proves the analogous expression for even centered moments. A representative denominator consists of an odd double factorial and an even-indexed Mersenne product:

$$M_{2n} = \frac{E_n}{(2n+1)!!\prod_{j=1}^{n}(2^{2j}-1)}.$$

The actual indexing in the source is chosen to align with its recursive sequence, but the structural point is the same. Odd moments vanish analytically, while even moments carry all arithmetic information.

The proof requires more combinatorial normalization than the half-moment case because binomial coefficients and parity restrictions appear in the recurrence. Factorial identities convert binomial coefficients into products compatible with the odd double factorial. The final division-free recurrence is again a natural-number equality.

9.7 Parity through Lucas' theorem

`Parity.lean` proves that the natural moment numerators are odd. The key combinatorial input is Lucas' theorem modulo two: a binomial coefficient $\binom{n}{k}$ is odd exactly when every one-bit of k occurs at a one-bit position of n. Consequently the number of odd entries in row n of Pascal's triangle is

$$2^{s_2(n)}.$$

The moment recurrence is reduced modulo two. Most terms vanish in pairs or carry explicit factors of two; the surviving binomial pattern is counted using Lucas' theorem. The source must bridge several representations of a finite range—natural inequalities, `Fin` types, and `Finset.range`—before applying mathlib's Lucas results. Dedicated cardinality lemmas hide this bridge from the final parity induction.

9.8 Reduced numerators, denominators, and two-adic valuation

`TwoAdic.lean` turns the normalization and parity theorems into exact assertions about reduced rational numerators and denominators. If a displayed quotient has odd numerator and a denominator whose power of two is known, then no cancellation of two-adic factors is possible. This determines the two-adic valuation of the reduced denominator.

The argument differs for odd and even indices. Odd-index statements reduce to the normalized even moments. Even-index statements use the half-moment recurrence together with the Lucas count of odd binomial coefficients. The formal proof includes lemmas relating:

- the numerator and denominator fields of a normalized rational;
- divisibility by two in $\mathbb{N}$ and $\mathbb{Z}$;
- coprimality in reduced form; and
- a valuation statement for the dyadic value $F(2^{-n})$.

9.9 Denominator bounds

`DenominatorBound.lean` and `HalfMomentDenominator.lean` assemble divisibility results into explicit bounds. Rather than attempting to compute the reduced denominator directly, the proof supplies a common denominator known to clear every term in a recurrence. The reduced denominator then divides that common denominator by the universal property of rational normalization.

This method is robust under changes of indexing and does not require unique factorization calculations for the full denominator. Exact two-adic theorems can be layered on top where a sharper result is needed.

9.10 Bernoulli recurrences and source-facing arithmetic

`BernoulliRecurrences.lean` translates some moment identities into Bernoulli-number language used by the arithmetic paper. The formalization keeps Bernoulli manipulations separate from the basic moment recurrence: the latter is elementary and executable, while the former uses a larger special-function and finite-sum API.

`PaperStatements.lean` then exposes propositions and theorems with numbering matching the paper. Its proofs are often short compositions of the stronger internal results described above. This is a good place to learn what each source theorem says, but not always the place to learn why it is true.

10 Fourier analysis and Rvachev's original characterization

10.1 Why the up-function is the Fourier object

The bounded Fabius function has nonzero constant tails and is not integrable on the line. The Rvachev up-function is compactly supported, smooth, and even, so it belongs to every L^p space and, after viewing it as a smooth function with compact support, defines a Schwartz function. This is the natural object for the Fourier-transform theorems in `FourierAnalytic.lean` and `OriginalUniqueness.lean`.

The original characterization asks for a smooth compactly supported function satisfying a refinement or derivative equation and a normalization. The repository formalizes both existence and uniqueness. The uniqueness proof is especially instructive because it turns a differential equation in physical space into a multiplicative recurrence in frequency space.

The source predicate retains the paper's explicit assumption that its dilation scale is positive. The smart constructor `IsOriginalFabius.mk_of_derivative_law` shows that this sign is actually forced by the remaining hypotheses: a mean-value point in $(-1, 0)$ reduces the derivative law to $k\varphi(2c + 1) = 1$ with a strictly positive second factor.

The two formal solution spaces are connected explicitly. The theorem `IsFabius.isOriginalFabius_rvachevUp` says that the Rvachev fold of every bounded Fabius solution satisfies the original compact-support characterization at scale two. Conversely, `isOriginalFabius_iff_existsUnique_isFabius` says that every original solution has scale two and is the fold of a unique bounded Fabius solution. This is the correct tail-free equivalence: a direct predicate iff for an arbitrary bounded function would be false because its fold never inspects values on $(1, \infty)$. The theorem `rvachevUp_eq_iff_eqOn_Iic_one` makes that loss of information exact: two folds agree if and only if their candidates agree on $(-\infty, 1]$. Accordingly, `isFabius_iff_isOriginalFabius_rvachevUp_and_rightTail` restores the sharp fixed-candidate iff by adjoining only the condition $F(x) = 1$ for $x > 1$.

10.2 Fourier conventions

Formal Fourier proofs are extremely sensitive to normalization. Mathlib's transform includes a fixed kernel convention; the source therefore writes explicit factors of π, 2π, and the imaginary unit rather than silently adopting a paper convention. The sinc factor is defined compatibly with that convention and extended continuously at zero.

A schematic transform identity is

$$\widehat{\mathrm{up}}(z) = \mathrm{sinc}(\pi z)\,\widehat{\mathrm{up}}(z/2).$$

Depending on whether the source theorem uses angular or ordinary frequency, the argument of sinc is adjusted. The Lean theorem's constants should be taken as authoritative; the important structural feature is multiplication by one sinc factor and halving of frequency.

10.3 Transforming the derivative equation

The source first proves that a candidate satisfying the original assumptions is compactly supported and smooth enough to be represented as a Schwartz map. It then applies the Fourier derivative theorem and the affine change-of-variable theorem. A derivative contributes a frequency factor; a dilation contributes both a reciprocal Jacobian and a rescaled frequency; translations contribute complex phases.

After simplification, the phases combine into a sine quotient, producing the sinc recurrence. The mass normalization gives

$$\widehat{\mathrm{up}}(0) = \int_{\mathbb{R}} \mathrm{up}(x)\ \mathrm{d}x = 1.$$

This is the seed value that controls the limit after infinitely many refinements.

Lean pattern. The proof does not rewrite Fourier integrals by hand at every step. It first establishes membership in the function class required by mathlib, then invokes library theorems for derivative, translation, and dilation. Only the final algebra of complex exponentials is expanded. This concentrates the fragile measure-theoretic assumptions near the beginning of the proof.

10.4 Iteration to a finite product

Iterating the refinement identity n times gives

$$\widehat{\mathrm{up}}(z) = \left(\prod_{j=0}^{n-1} \mathrm{sinc}\left(\frac{\pi z}{2^j}\right)\right) \widehat{\mathrm{up}}\left(\frac{z}{2^n}\right).$$

This is proved by induction. The Lean induction must reconcile two equivalent ways of writing a finite product: appending the last factor versus splitting off the first. A small library of `Finset.range` product identities keeps the main theorem readable.

As $n \to \infty$, $z/2^n \to 0$, so continuity of the Fourier transform gives

$$\widehat{\mathrm{up}}(z/2^n) \to 1.$$

The formal product proof uses the available bound $\mathrm{complexSinc}(w) - 1 = O(w)$ near zero, so the deviations are dominated by the geometric series $\sum_j |\pi z|/2^j$. The sharper classical quadratic expansion is not needed.

10.5 The infinite sinc product

`FourierProduct.lean` develops the technical product independently. It proves that complex sinc is entire, establishes pointwise multipliability and the finite-product recurrence, and identifies the product with the analytic Fourier transform. Continuity then follows through that identification rather than from a separate locally uniform product theorem in this module.

Infinite products in Lean can be represented through partial products and limits rather than through a single primitive operator. The source proves that the sequence is Cauchy using a summable bound on deviations from one. Special care is taken at zeros of a sinc factor: logarithmic product arguments are invalid there, whereas direct partial-product convergence remains sound.

The result is the celebrated formula

$$\widehat{\mathrm{up}}(z) = \prod_{j \geq 0} \mathrm{sinc}\left(\frac{\pi z}{2^j}\right)$$

with the indexing adjusted to the normalization used by the code.

10.6 Fourier injectivity and uniqueness

If two original candidates satisfy the same hypotheses, the iteration argument gives the same Fourier transform for both. The source then uses injectivity of the Fourier transform on Schwartz functions to conclude equality. The final transition is:

1. package each smooth compactly supported function as a Schwartz map;
2. prove their transforms are equal as functions;
3. apply Schwartz-space Fourier injectivity; and
4. project the equality back to ordinary functions.

The theorem also shows that the scaling constant in the original functional equation is forced, rather than being an arbitrary parameter. Integrating the derivative law on $[-1, 0]$ and using the mass normalization leaves only the canonical value, which in the repository's normalization is two.

10.7 Moment series from the Fourier product

Expanding the transform at zero recovers moments. The logarithm of the sinc product yields cumulant-like sums, while direct multiplication gives a power series whose coefficients are rational combinations of powers of two. `FourierAnalytic.lean` supplies the entire transform and inversion interface; the moment-series identification itself is proved in `AnalyticMoments.lean`.

The proof again prefers an identity of entire functions followed by coefficient comparison. Establishing an entire power series permits differentiation at zero to extract moments without repeatedly justifying differentiation under the integral sign.

10.8 Poisson summation

The original paper's Poisson-summation theorem is formalized in a dedicated module. Compact support and smoothness place the function in the Schwartz class, so mathlib's Poisson summation framework applies. The main work is normalization: the theorem from the library and the paper use different Fourier conventions and summation coordinates. The proof rewrites the transform through the sinc product and then simplifies the sampled factors.

For positive spacing u, the normalized identity has the form

$$\sum_{k \in \mathbb{Z}} \mathrm{up}(t + uk) = \frac{1}{u} \sum_{k \in \mathbb{Z}} \widehat{\mathrm{up}}(k/u) e^{2\pi \mathrm{i} kt/u}.$$

The formal audit records two source corrections visible in this equation: the printed equation (25) omitted the translation variable t from the exponential, and a later displayed $1/a$ must disappear after both sides are multiplied by a.

The same Schwartz realization yields more than the summability needed here. The public theorem `rvachevFourier_real_iteratedDeriv_rapidDecay` bounds every real-axis derivative after multiplication by an arbitrary power of the frequency, while `rvachevFourier_real_rapidDecay` is the transform-only specialization. Thus the rapid decay invoked by Poisson summation is itself part of the reusable Lean API.

Compact support sharpens the $t = 0$ specialization on the full ray $a \geq \frac{1}{2}$. Only the lattice sites $-1, 0, 1$ remain, so

$$\sum_{m\in\mathbb{Z}} \operatorname{up}(am) = 1 + 2\operatorname{up}(a), \qquad a + 2a\operatorname{up}(a) = \sum_{m\in\mathbb{Z}} \widehat{\operatorname{up}}(m/a).$$

The declarations `rvachev_lattice_sum_of_one_half_le` and `rvachev_poisson_support_specialization_of_one_half_le` require no upper bound $a \leq 1$; older two-bound declarations remain compatibility wrappers.

This is representative of source-facing modules: the analytic theorem already exists in a reusable library form, while the paper theorem is a carefully normalized corollary.

Proof architecture. The Fourier uniqueness proof is a rigidity argument. The differential/refinement equation repeatedly pushes all unknown information into the value of the transform at a frequency tending to zero. Normalization fixes that limiting value, so no freedom remains. □

11 Finite convolutions, probability, and approximation

11.1 The probabilistic representation

`ProbabilityRepresentation.lean` uses the product probability space $\Omega = [0,1]^{\mathbb{N}}$ and the geometrically weighted coordinate sum

$$X(\omega) = \sum_{n\geq 0} \frac{\omega_n}{2^{n+1}}.$$

Splitting off the first coordinate gives the exact self-similarity

$$X(\omega) = \frac{\omega_0 + X(\operatorname{tail}\omega)}{2}.$$

If $H(y) = \mathbb{P}[X \leq y]$, conditioning on ω_0 produces

$$H(y) = \int_0^1 H(2y - u)\,\mathrm{d}u.$$

The module proves globally that $H = F$, and identifies the folded function by

$$\operatorname{up}(x) = H(1 - |x|).$$

This is the direct CDF model on $[0, 1]$. It should not be conflated with the centered finite-convolution measures used by the original-paper approximants.

The self-similarity of X is the probabilistic counterpart of the fixed-point integral equation. It supplies a second route to the same canonical bounded function once uniqueness of the `IsFabius` specification is available.

11.2 Finite convolution measures

A formal convolution proof must verify that every measure involved is finite, usually a probability measure. The source defines the elementary measures, proves their support and total mass, and then uses induction to show the convolution remains a probability measure. Associativity is applied only after the necessary sigma-finiteness or finiteness instances are in scope.

The support of the n-fold convolution is the Minkowski sum of the elementary supports. Geometric decay keeps these supports in a fixed compact interval. This uniform support bound supplies tightness essentially for free and gives uniform moment bounds.

11.3 Weak convergence

`WeakConvergence.lean` proves convergence of the finite convolution measures by first establishing pointwise convergence of their characteristic functions and then applying mathlib's Lévy continuity theorem. It derives convergence against bounded continuous test functions as the reusable public form needed by later interval and distribution-function arguments.

The limiting density measure `rvachevMeasure` has exact topological support $[-1, 1]$: every open set meeting $(-1, 1)$ has positive mass, and the endpoints belong to the support even though the density vanishes there.

The test-function formulation avoids trying to prove pointwise convergence of densities too early. Weak convergence is stable under convolution and directly reflects the probabilistic construction. To recover the Fabius function, the code later specializes to approximations of interval indicators.

11.4 Step approximants

`StepApproximationLimit.lean` studies the explicit step functions arising from finite convolution coefficients. The coefficient arrays satisfy symmetry and unimodality properties. These imply monotonicity of the approximants on each side of the center and provide pointwise upper and lower controls.

The desired limit is continuous, but indicator functions of intervals are not continuous test functions. The source uses an interval-average squeeze. Integrals over a small neighborhood converge by weak convergence; monotonicity bounds the point value of a step function between nearby interval averages. Shrinking the neighborhood and using continuity of the limit gives pointwise convergence.

11.5 The endpoint half-weight correction

Closed interval indicators double-count shared endpoints when adjacent steps are assembled. In ordinary integration this affects a measure-zero set, but a pointwise theorem about the approximating function sees the discrepancy. The formalization therefore uses a representative with half weight at step boundaries.

This correction is a good example of Lean revealing a genuine mathematical issue rather than a mere elaboration problem. The source paper's almost-everywhere intuition is adequate for an integral identity, but not for the stated pointwise approximation. Once the half-endpoint convention is adopted, neighboring intervals partition correctly and the convergence theorem has the intended endpoint values.

11.6 Coefficient unimodality

The finite convolution coefficients form rows with strong symmetry and unimodality. The proof proceeds through a recurrence: the next row is obtained by averaging shifted blocks of the previous row. Symmetry is immediate from the symmetric elementary measure; monotonicity requires comparing overlapping partial sums.

Lean handles the row as a finite function or list. The proof establishes index bounds before rewriting recurrence entries. Boundary entries, where one shifted block leaves the valid range, are split into separate cases. These technical cases explain why the coefficient lemmas occupy a substantial part of `StepApproximationLimit.lean`: the final analytic squeeze is comparatively short once monotonicity is available.

11.7 From distribution functions to the bounded Fabius function

The direct weighted sum $X \in [0,1]$ from `ProbabilityRepresentation.lean` has CDF F globally. The folded identity is stated directly as

$$\mathrm{up}(x) = \mathrm{weightedSumCDF}(1 - |x|) = \mathbb{P}[X \le 1 - |x|].$$

These are CDF and event-probability theorems; the module does not package a separate centered random variable or density theorem.

This gives an independent conceptual construction of the same object as the Banach fixed point. In the formal dependency graph it is normally developed after the canonical function is available, because identification is easier than rebuilding the entire `IsFabius` interface from measure convergence. Mathematically, however, it explains why the function is a distribution function.

11.8 Exact finite Taylor formulas and nonanalyticity

`OriginalPaperSupplement.lean` packages further theorems from the original paper. At centered dyadic points, sufficiently high derivatives vanish on one side because repeated scaling reaches the support boundary. Taylor's theorem becomes an exact finite polynomial identity on a small interval.

If the function were analytic at such a point, its Taylor series would terminate and the function would be a polynomial in a neighborhood. The differential equation and support properties rule this out unless the function were trivial. Thus the exact local truncation contributes to a nowhere-analyticity theorem.

The formal proof distinguishes several notions:

- `ContDiff` smoothness;
- existence of derivatives of all orders at a point;
- equality to the Taylor series on a neighborhood; and
- mathlib's analytic-at predicate.

A proof that merely observes all derivatives vanish at a boundary point establishes flatness, not by itself nonanalyticity everywhere. The module supplies the additional translation and dyadic-density argument needed for the global conclusion.

11.9 Corrections recorded by the original-paper aggregator

`Paper05442.lean` and its supplement explicitly document several source-sensitive points. Among them are:

- finite convolution must be stated at the finite stage rather than with an already limiting object;
- step-function endpoints require half weights for pointwise identities;
- a factor of the transform variable is needed in one displayed Fourier equation;
- a formula involving a positive order requires the hypothesis $n > 0$; and
- a global dyadic identity needs the signed extension or an adjusted normalization.

These are not hidden as comments in low-level proofs. The aggregator treats them as part of the formal audit of the paper.

12 The two paper formalizations as navigable APIs

12.1 The first paper: `Paper05442.lean`

The module named after arXiv:1702.05442 imports the original uniqueness proof, probability representation, weak convergence, step approximation, Poisson summation, and supplementary source theorems. Its purpose is to certify coverage of the paper as a whole.

A useful reading order is:

1. `OriginalCharacterization.lean` and `OriginalUniqueness.lean` for the defining characterization and the equivalence of solution spaces;
2. `ProbabilityRepresentation.lean` for the random-series model;
3. `WeakConvergence.lean` for the measure limit;
4. `StepApproximationLimit.lean` for pointwise approximation;
5. `PoissonSummation.lean` for the sampled Fourier identity; and
6. `OriginalPaperSupplement.lean` for moments, Taylor formulas, rationality, and nonanalyticity.

The aggregator's theorem names are source-facing, but the mathematical objects remain those of the common core. This avoids a second, paper-specific definition of the function.

12.2 The arithmetic paper: `Paper06487.lean`

The second aggregator, named after arXiv:1702.06487, imports `Paper06487Supplement.lean`, which in turn draws on `PaperStatements.lean` and the arithmetic/analytic bridge modules. It covers the paper's propositions on moments, exact dyadic values, denominators, parity, and two-adic valuations.

The internal proof structure is more extensive than the paper numbering suggests:

$$\text{exact recurrence} \longrightarrow \text{normalized numerator} \longrightarrow \text{parity/divisibility} \longrightarrow \text{reduced rational theorem}.$$

Each arrow is represented by one or more modules. The final paper theorem is often a cast or notation wrapper around an internal theorem with a stronger domain or a more explicit divisibility conclusion.

12.3 Statements that remain questions or conjectures

A formalization should not turn an open question into a theorem. `PaperStatements.lean` records the paper's Question 5, Definition 12, and Conjecture 16 as definitions or propositions expressing the claimed predicate, not as proofs of the unresolved assertion. This is part of the source-coverage discipline: every numbered item is represented, but its logical status is preserved.

The surrounding proved theorems may establish initial cases, necessary conditions, or denominator bounds relevant to the conjecture. Their names make clear when the result is a theorem about the conjectural predicate rather than a proof of the conjecture itself.

12.4 Why a separate coverage document matters

The repository's `PAPER_COVERAGE.md` maps source theorem numbers to Lean declarations. This is more than project management. It helps distinguish three questions:

1. Has the mathematical claim been formalized?
2. Where is the strongest internal theorem proved?
3. Which wrapper corresponds to the source numbering and notation?

Appendix B reproduces the navigational content of that map in compact form and adds the surrounding module families.

13 Thue–Morse and q-binomial identities

13.1 Why q-binomial algebra appears

The Thue–Morse sign converts binary digit structure into alternating sums. When a finite Taylor formula is applied repeatedly across dyadic cells, the resulting coefficients naturally involve Gaussian, or q-binomial, coefficients at $q = 1/2$. The repository develops this relationship in a series of modules ranging from exact finite identities to global analytic series.

The core finite identity has the flavor

$$\sum_{k=0}^{2^n-1} \varepsilon(k)(x+k)^m,$$

which vanishes for low degree and factors in a controlled way at the critical degree. q-binomial coefficients provide a compact description after binary decomposition. The Fabius values enter when the polynomial identity is inserted into a scaled finite Taylor expansion.

The same even/odd recursion gives the exact exponential factorization formalized in `ThueMorseExponential.lean`:

$$\sum_{r<2^n} (-1)^{s_2(r)} e^{rz} = \prod_{j<n} \left(1 - e^{2^j z}\right).$$

It is a finite identity, so no interchange of limits or infinite products is hidden in the proof.

13.2 The finite q-Pochhammer infrastructure

`HalfQBinomial.lean` develops exact q-binomial algebra at $q = 1/2$ over $\mathbb{Q}$. The finite q-Pochhammer product is represented without analytic convergence issues:

$$(a;q)_n = \prod_{j=0}^{n-1} (1 - aq^j).$$

At $q = 1/2$, clearing powers of two turns many factors into Mersenne numbers. This is why the Mersenne products introduced in `Arithmetic.lean` reappear.

The module defines a rational `halfQBinomial` and proves its q-Pascal recurrence. It then proves a finite q-binomial theorem by induction. The induction step is not a black-box appeal to an imported polynomial theorem; it matches the precise normalization required downstream. Summands are shifted and paired so the q-Pascal recurrence telescopes into the next product.

13.3 Why prove a specialized q-binomial theorem?

Mathlib contains general polynomial and combinatorial infrastructure, but a specialized theorem at $q = 1/2$ gives the repository:

- exact rational coefficients with predictable denominators;
- direct compatibility with existing Mersenne products;

- a recurrence usable by kernel computation; and
- statements in the same finite-sum form as the Fabius formulas.

A general theorem would still require a substantial specialization layer. Proving the specialized identity directly keeps casts and nonvanishing side conditions under control.

13.4 Raw, scalar, and polished formulas

The q-binomial family is intentionally split into several modules:

- `FabiusRawQBinomialFormula.lean` and `FabiusRawQBinomialScalar.lean` establish algebraic sums before all Fabius-specific simplifications;
- `FabiusQBinomialFormula.lean` proves the exact finite identity in its principal form;
- `FabiusQBinomialScalarFormula.lean` specializes coefficient-valued statements to scalar values; and
- `FabiusDyadicQBinomialScalar.lean` inserts the exact dyadic evaluator.

This decomposition protects the core polynomial identity from unnecessary analytic dependencies. It also allows translation invariance to be proved at the polynomial level and reused after evaluation.

13.5 The unrestricted finite identity

A notable strength of the formal result is that it is not restricted to a reduced dyadic representative or to $0 \le m \le 2^n$. It is stated for arbitrary natural parameters in the range supported by the algebraic expression. The signed global extension absorbs the translation behavior that would otherwise force an artificial range hypothesis.

The proof proceeds by induction on the binary depth. Splitting a range into even and odd indices transforms the Thue–Morse sign into a difference of two scaled copies. The induction hypothesis applies to each copy, and the q-Pascal recurrence combines the coefficients. Polynomial degree bounds control vanishing terms.

Proof architecture. The even/odd split is the common engine behind Thue–Morse cancellation and q-binomial recursion. In Lean, the decisive step is a reindexing theorem that expresses a sum over `Finset.range(2*N)` as sums over $2k$ and $2k+1$. Once that lemma has the right shape, the sign and power transformations simplify automatically. □

13.6 Translation invariance as a polynomial theorem

One of the strongest internal statements says that the relevant finite q-expression is independent of a translation parameter because it is a polynomial of degree below the cancellation threshold. This is proved before specializing the parameter to real or complex values. As a result, common translation invariance holds over a broad coefficient ring and later applies uniformly to $\mathbb{R}$ and $\mathbb{C}$.

This is a recurring proof-design strategy: prove a fact at the most algebraic level where it is true, then transport it through evaluation homomorphisms. Analytic specializations become one-line consequences of polynomial extensionality.

13.7 Polynomial constancy and Taylor reduction

In `FabiusQBinomialTaylor.lean`, q is a common translation indeterminate, not an asymptotic expansion parameter. Complete-block Thue–Morse cancellation proves that the finite numerator is a constant polynomial in q. Evaluation is therefore independent of the translation over every characteristic-zero field. After normalization and reindexing, the constant is identified exactly with `fabiusReductionSum`.

This algebraic constancy is distinct from the discrete-limit approximation `fabiusDiscreteLimitApproximationKqxn`. A finite approximation there can genuinely depend on a complex q; later complex-shift estimates prove only that this dependence vanishes in the prescribed limit.

13.8 The global binary-reduction series

`FabiusBinaryReductionSeries.lean` and `FabiusGlobalQBinomialSeries.lean` pass from finite identities to an infinite series representing the signed extension. The binary reduction follows the same highest-bit philosophy as the exact evaluator, but now the steps are organized as a global q-series.

A genuine scale-zero term forces the outer index to start at $m = 0$. Its previous prefix is the half-scale floor $\lfloor x/2 \rfloor$, so the zeroth coefficient distinguishes even and odd unit blocks of the signed extension. It vanishes for $x < 1$ and equals one at $x = 1$; hence the older $m = 1$ presentation survives only on $0 \le x < 1$.

The exact finite telescope leaves a signed residual whose uniform geometric bound is at most 2^{1-N} through inclusive scale $N \ge 1$, for every real input. It therefore tends uniformly to zero on the whole real line. The summand norms are pointwise summable, giving all-real `HasSum` and `tsum` identities. `FabiusGlobalQBinomialSeries.lean` proves that its literal real or complex q-summand is pointwise equal to the q-independent binary-reduction summand before either side is summed. The resulting real and complex series represent the signed extension for every $x \in \mathbb{R}$; the earlier nonnegative-input statements remain compatibility wrappers.

13.9 Parity-power series

`FabiusParityPowerSeries.lean` rewrites the corrected scale-zero binary-reduction series in the draft's parity-power notation. The outer sum starts at $m = 0$ and equals the signed global Fabius extension for every $x \in \mathbb{R}$, with the nonpositive side termwise zero; the original series starting at $m = 1$ is recovered only on $0 \le x < 1$. Its integer exponent is $\kappa_n = \binom{n+1}{2} - 1$, so $\kappa_0 = -1$, not a truncated natural subtraction; the scale-zero identity also uses Lean's convention $0^0 = 1$. The proof identifies the inner finite sum with one half of `fabiusReductionSum` and then reuses the binary telescope.

14 Discrete limits and complex shifts

14.1 The shape of the limit theorem

The discrete-limit project starts from finite q-binomial/Thue–Morse rows depending on a real location x and a complex parameter q. For nondyadic x, an individual finite row can depend genuinely on q. The theorem is that, after the prescribed normalization and as the row depth tends to infinity, the limit exists and is independent of q, equaling the relevant Fabius expression.

This distinction between finite-stage dependence and limiting independence is explicitly formalized. It prevents an overstrong claim that each approximant is constant in the parameter.

14.2 Uniform spline approximants

`FabiusUniformSpline.lean` introduces centered finite Thue–Morse splines. These are piecewise-polynomial functions whose coefficients encode a finite convolution row. They are uniformly bounded, vanish on the nonpositive half-line, and repeat on the nonnegative axis as signed two-unit blocks. `FabiusComputability.lean` proves the global error estimate

$$\sup_{x \in \mathbb{R}} |S_p(x) - \mathcal{F}(x)| \le 2^{-p} \qquad (p \ge 1).$$

Thus they converge uniformly on all of $\mathbb{R}$, while their bounded-CDF specialization converges uniformly to F on $[0,1]$.

On the fundamental interval the source identifies the spline with the CDF of a centered finite weighted sum. This yields monotonicity, range and support bounds, and pointwise convergence; a block-translation identity extends those facts across the nonnegative axis.

14.3 Complex-shift control

`FabiusComplexShiftSpline.lean` and `FabiusDiscreteLimitComplexShift.lean` control evaluating a finite spline at a complex displacement. A Taylor expansion separates the real centered value from correction terms. Uniform derivative bounds, together with the scale of the shift, show the corrections vanish in the limit.

The proof is careful not to use order arguments on $\mathbb{C}$. Norm bounds replace real inequalities, and factorial denominators are estimated in $\mathbb{R}$ before being cast into complex norms. The finite nature of the Taylor expansion is again important: each finite spline has bounded degree, so no complex analytic remainder theorem is required.

14.4 The Toeplitz averaging mechanism

`FabiusDiscreteLimitToeplitz.lean` rewrites the normalized row as a Toeplitz-type weighted average of centered spline values. The weights need not be nonnegative: their row sum is one and their total variation is uniformly bounded, while the sampled spline degrees all tend to infinity. These facts transfer convergence of the splines to convergence of the averages.

The finite prefix is encoded safely as the natural floor

$$\texttt{fabiusDiscreteLimitRangeLength}(x,p) = \left\lfloor 2^p x + \tfrac{1}{2} \right\rfloor_+,$$

so no negative length is coerced into a natural number. For $x \geq 0$, its integer cast is $\lfloor 2^p x - \frac{1}{2} \rfloor + 1$, and the length is zero exactly when $2^p x < \frac{1}{2}$. The exact row-mass theorem gives sum 1. A lower bound $\mathrm{halfQPochhammer}(n) \geq 1/4$ yields the explicit total-variation bound 16, independent of the row. Finally, the sampled indices are shown to escape every fixed finite set uniformly. These are the concrete inputs to the abstract Toeplitz passage; positivity of the weights is neither assumed nor needed.

The generic theorem `tendsto_weighted_rows_of_tendsto` isolates four hypotheses:

1. every finite row has sum 1;
2. the row total variations have one uniform bound;
3. every sampled index in a row eventually lies beyond each prescribed cutoff; and
4. the sampled sequence $H(n)$ tends to its proposed limit.

No positivity or order structure on the coefficient field is required. This abstraction is useful beyond the specific Fabius application and keeps the final limit proof from becoming a nested epsilon argument. Searchable entry points are `sum_range_discreteLimitWeight`, `one_fourth_le_halfQPochhammer`, `sum_abs_discreteLimitWeight_le`, and `tendsto_weighted_rows_of_tendsto`.

14.5 Final reindexing and identification

`FabiusDiscreteLimitIntegration.lean` performs the exact outer q-binomial reindexing and applies the Toeplitz theorem to the complex-shift spline limit. It identifies the result with the signed global Fabius function and with the previously proved binary and literal q-binomial series. The main nonnegative-input theorem is strengthened by exact empty-prefix vanishing to every real input.

14.6 Proof flow of the discrete-limit family

A recommended source order is:

1. `FabiusRawQBinomialFormula.lean`;
2. `FabiusQBinomialFormula.lean`;
3. `FabiusBinaryReductionSeries.lean`;
4. `FabiusUniformSpline.lean`;
5. `FabiusComplexShiftSpline.lean`;
6. `FabiusDiscreteLimitToeplitz.lean`;
7. `FabiusDiscreteLimitComplexShift.lean`; and
8. `FabiusDiscreteLimitIntegration.lean`.

The sequence moves from exact finite algebra, through real approximation, to complex perturbation, averaging, and final analytic identification.

Formalization-sensitive point. When reading these files, distinguish the row index from the polynomial degree, the dyadic scale, and the q-parameter. Informal drafts often use a single letter for more than one role. The Lean development uses separate variables and explicit type annotations, which can make theorem statements longer but prevents illegal substitutions.

---

15 The logarithmic scale near the flat endpoint

15.1 Why ordinary Taylor asymptotics fail

Every derivative of the bounded Fabius function vanishes at zero, yet $F(x) > 0$ for every $x > 0$. Thus no nonzero power of x is the leading term. The correct scale is logarithmic and, at first order, quadratic in $-\log x$.

The asymptotic development introduces

$$\phi(t) = F(2^{-t}), \qquad g(t) = -\log \phi(t).$$

As $x \downarrow 0$, the variable $t = -\log_2 x$ tends to infinity. This turns dyadic scaling into unit translation and is the natural coordinate for the delay equation.

15.2 The exact logarithmic delay identity

`FabiusLogScale.lean` differentiates $\phi(t) = F(2^{-t})$. For $t \geq 1$, the argument 2^{-t} lies in the left branch where the differential equation applies. The chain rule gives

$$\phi'(t) = -(\log 2)2^{-t}F'(2^{-t}) = -2(\log 2)2^{-t}F(2^{1-t}).$$

Since $\phi(t-1) = F(2^{1-t})$, this becomes a delay relation. Rewriting in terms of $g = -\log \phi$ and its derivative produces the exact identity formalized in the module:

$$g(t) - g(t-1) = \log g'(t) - \log(\log 2) - (1-t)\log 2, \qquad t \geq 1.$$

The source first proves positivity of $F(2^{-t})$ and the relevant logarithmic slope, so every logarithm is legal.

Formalization-sensitive point. The threshold $t \geq 1$ is essential. The bounded function's derivative law does not hold on the same branch for all real t. An informal draft treated the delay identity as global; the Lean theorem records the correct domain and downstream asymptotic arguments are formulated with eventual filters.

15.3 Positivity near zero

Taking a logarithm requires strict positivity, not merely nonnegativity. The repository obtains positivity from monotonicity and exact dyadic values. Given $x > 0$, choose a sufficiently large inverse power of two below x. Exact arithmetic shows $F(2^{-n}) > 0$, and monotonicity gives $F(x) \geq F(2^{-n}) > 0$.

This proof is representative of the interaction between exact and analytic layers: a qualitative analytic property at every positive real point is obtained from exact values on a dense cofinal sequence.

15.4 The quadratic logarithmic law

`FabiusLogSquaredAsymptotic.lean` proves

$$\frac{g(t)}{t^2} \longrightarrow \frac{\log 2}{2}.$$

Equivalently,

$$\log F(x) \sim -\frac{(\log(1/x))^2}{2\log 2} \qquad (x \downarrow 0).$$

The proof first establishes sufficiently sharp estimates at natural values of t, where the exact inverse-dyadic recurrence is available. It then extends them to real t by monotonicity and floor/ceiling comparison.

If $n \leq t < n+1$, monotonicity gives

$$g(n) \leq g(t) \leq g(n+1)$$

with the inequality direction determined by the decreasing behavior of $F(2^{-t})$. Dividing by t^2 and using $n/t \to 1$ transfers the natural asymptotic to real arguments.

15.5 A quantitative coarse bound

The same module proves a bound of the form

$$\left| g(t) - \frac{\log 2}{2} t^2 \right| \leq C t \log t$$

for all sufficiently large t. This is weaker than the eventual sharp formula but strong enough to establish flatness and compare F with elementary scales.

Such a concrete inequality is more useful in Lean than a bare $o(t^2)$ statement. It can be inserted into exponential comparisons with explicit thresholds, and it supports later proofs that every power of x dominates $F(x)$ while every $e^{-c/x}$ is dominated by $F(x)$.

15.6 Filter formulations

The repository states asymptotics both in the $t \to \infty$ coordinate and as $x \to 0^+$. Conversion lemmas relate the filters under $x = 2^{-t}$. A typical proof establishes a limit along `atTop`, composes with the continuous map $t \mapsto 2^{-t}$, and then identifies the image filter with `nhdsWithin0(Set.Ioi0)`.

Explicit conversion lemmas are preferable to ad hoc substitutions in each theorem. They ensure that positivity domains and logarithm definitions are carried through uniformly.

16 Negative Laplace analysis and the periodic correction

16.1 The negative moment generating function

The repository's entire generating function is totalized as

$$G(z) = 1 + z\int_0^1 \mathrm{up}(t)e^{zt}\,\mathrm{d}t.$$

For $s > 0$, `NegativeLaplace.lean` identifies its negative-axis value with the exact product

$$e^{\mathrm{negativeLaplaceLog}(s)} = G(-s) = \prod_{j\ge 1} \frac{1-e^{-s/2^j}}{s/2^j}.$$

The companion `LaplaceTransform.lean` proves the equivalent bounded-function formula

$$\frac{G(-s)}{s} = \int_{(0,\infty)} F(t)e^{-st}\,\mathrm{d}t.$$

The product's logarithm is

$$L(s) = \sum_{j\ge 1}\left[\log(1-e^{-s/2^j}) - \log(s/2^j)\right].$$

`NegativeLaplace.lean` constructs this series carefully and proves absolute convergence after grouping terms in a form adapted to small and large arguments.

16.2 The kernel and its bounds

The central scalar kernel is the logarithm of a normalized Bose factor. For small u, the quotient $(1-e^{-u})/u$ tends to one, so its logarithm is $O(u)$. For large u, the exponentially small correction $\log(1-e^{-u})$ is controlled by a geometric series.

The source proves elementary inequalities before summing over dyadic scales. Typical estimates use

$$0 \le -\log(1-v) \le \frac{v}{1-v} \qquad (0 \le v < 1),$$

with $v = e^{-u}$. The resulting majorants are summable after the dyadic split at the scale where $s/2^j$ crosses one.

16.3 The exact dilation equation

The product shifts cleanly under $s \mapsto 2s$. Removing the first factor and reindexing the rest gives an exact functional equation for $L(s) = \log G(-s)$. Solving the associated difference equation in the logarithmic variable $u = \log_2 s$ produces a quadratic polynomial plus a periodic remainder.

The formal decomposition is

$$L(s) = -\frac{(\log s)^2}{2\log 2} + \frac{1}{2}\log s + R(\log_2 s) + E(s),$$

where $R(u+1) = R(u)$ and the forward-tail error satisfies an explicit exponential bound. A representative bound proved in the module is

$$|E(s)| \le \frac{e^{-s}}{(1-e^{-s})^2},$$

and for s beyond a fixed threshold this is simplified to a constant multiple of e^{-s}.

The sign information is equally exact on the positive half-line. Every logarithmic factor and the full logarithmic product are strictly negative, as is the forward tail. Consequently `negativeLaplaceTailError`, defined as the negative of that tail, is strictly positive and its absolute value can be removed.

The reusable differentiation step is isolated in `ScalingRecurrence.lean`, a Mathlib-only module independent of the Fabius definitions. It differentiates multiplicative scaling recurrences for an arbitrary dilation $q > 0$, in both vector-valued and real-scalar forms, with the outgoing weight normalized by the caller. Its inert-parameter version acquires no factor of q, because differentiation is in the other variable; dyadic wrappers feed these results into the ordinary and vertical negative-Laplace derivative layers.

16.4 How the periodic function is constructed

The periodic term is not guessed from numerics. The code builds it by comparing the exact logarithm with a particular quadratic solution of the dilation equation. The difference is invariant under doubling up to a forward tail. Iterating the dilation toward infinity makes that tail summable; adding its convergent correction yields an exactly invariant function of $\log_2 s$.

Concretely, one introduces a defect

$$D(s) = L(s) + \frac{(\log s)^2}{2\log 2} - \frac{1}{2}\log s.$$

The dilation identity shows that $D(2s) - D(s)$ is an exponentially small kernel. A forward sum of these defects corrects D into a function satisfying $P(2s) = P(s)$. Writing $R(u) = P(2^u)$ gives period one.

Proof architecture. The periodic correction is the homogeneous solution of a dyadic difference equation. The quadratic logarithmic terms form a particular solution. Exponentially small forward tails determine which periodic solution corresponds to the actual infinite product. □

16.5 Continuity and regularity

`PeriodicCorrection.lean` proves continuity of R. The defining correction is a locally uniformly convergent series on positive compact intervals. The proof covers a compact interval $[a, b] \subset (0, \infty)$, bounds every tail term uniformly by a summable geometric sequence depending on a, and applies a Weierstrass-type theorem.

Later modules strengthen regularity. `PeriodicRegularity.lean` differentiates the correction series term by term after proving uniform convergence of derivative tails. `PeriodicMean.lean` computes or normalizes its mean. `PeriodicFourier.lean` identifies Fourier coefficients through Mellin-transform data.

16.6 Mellin–Bose analysis and finite parts

`MellinBose.lean` proves, first for real $a > 0$,

$$\int_0^\infty x^{a-1}\log(1-e^{-x})\,\mathrm{d}x = -\Gamma(a)\zeta(a+1),$$

by expanding the logarithm into an absolutely integrable exponential series and integrating term by term. `MellinFinitePart.lean` regularizes the double pole of $\Gamma(s)\zeta(1+s)$ at $s = 0$, while `BoseFinitePartIntegral.lean` realizes the finite part as convergent split integrals after subtracting the small-x singular term.

The local constants are normalized explicitly. `GammaSecondOrder.lean` proves the sharp complex remainder

$$\Gamma(1+z) = 1 - \gamma z + \left(\frac{\gamma^2}{2} + \frac{\pi^2}{12}\right) z^2 + O(z^3) \qquad (z \to 0),$$

while `StieltjesConstant.lean` fixes γ_1 by declaring the derivative of `zetaRegularReal` at one to be $-\gamma_1$, recorded in `HasDerivAt` form. Thus the sign of the linear coefficient in $\zeta(1+s) = s^{-1} + \gamma - \gamma_1 s + O(s^2)$ is part of the checked interface.

`PeriodicFourier.lean` treats the zero mode through the mean calculation and the nonzero modes by the substitution $x = 2^t$, dyadic interval decompositions, an identity-theorem extension to the open right half-plane, and continuity to the punctured imaginary axis. It obtains the Gamma–zeta Fourier coefficients and absolute Fourier reconstruction without a residue calculation, contour truncation, or Mellin inversion.

16.7 Why the periodic term cannot be dropped

The oscillatory correction $R(\log_2 s)$ is bounded but nonconstant. Therefore it is lower order than the quadratic logarithmic term but not an error tending to zero. Any asymptotic formula claiming an $o(1)$ or $O(1/\log s)$ residual after omitting R is false.

`FabiusWikipediaObstruction.lean` and related audit modules formalize this obstruction. They prove nonconstancy through a nonzero Fourier coefficient or a pair of points with unequal values. Sampling along two dyadic phase subsequences then produces distinct limit points for the alleged residual.

16.8 Identification with the generating function

After constructing the product and its logarithm, `NegativeLaplace.lean` proves equality with the analytic generating function defined earlier. The route is:

1. establish the product formula for finite truncations;
2. prove convergence to the infinite product;
3. establish the same functional equation and normalization for the generating function; and
4. use uniqueness or direct limit passage to identify them.

This closes the loop between exact moment series, Fourier/probability products, and the asymptotic Laplace object.

17 Bromwich inversion and the sharp saddle point

17.1 From a transform to the endpoint density

For every $r > 0$, `FabiusBromwichInput.lean` proves the exact Fourier–Bromwich identity

$$F(x) = e^{rx} \int_{\mathbb{R}} e^{2\pi i \xi x} \, \frac{G(-(r+2\pi i\xi))}{r + 2\pi i\xi} \, d\xi.$$

Thus the vertical coordinate $z = r + 2\pi i\xi$ has positive real part, while the generating function is sampled at the negative argument $-z$. The theorem comes directly from Fourier inversion of the exponentially tilted CDF and requires no contour-shifting assumption.

`BromwichSaddle.lean` supplies the generic quantitative framework and `FabiusBromwichInput.lean` discharges its Fabius-specific integrability and decay hypotheses. For asymptotic extraction one then chooses the explicit lower-Lambert reference radius and splits the resulting real integral into central and tail regions.

17.2 The Lambert phase

The dominant balance introduces a lower-Lambert reference coordinate. On its natural small-positive-x domain the repository defines $\lambda(x)$ so that

$$\lambda(x)\,2^{-\lambda(x)} = x.$$

Equivalently, if $x = 2^{-t}$, then

$$\lambda(2^{-t}) - \log_2 \lambda(2^{-t}) = t.$$

These are exact algebraic identities for the chosen coordinate. The formal module deliberately does not claim that this reference coordinate is the exact critical point of every residual term in the full contour phase.

The advantage of retaining the exact Lambert phase is that all-order expansions can be stated without repeatedly composing truncated asymptotic series. The leading behavior is

$$\lambda(2^{-t}) = t + \log_2 t + o(1),$$

or the corresponding expression after substituting $t = -\log_2 x$. The exact relation captures the corrections needed in both the exponential and Gaussian normalization.

`FabiusLambertPhase.lean`, `FabiusLambertRates.lean`, and `FabiusLambertSaddle.lean` establish the domain, monotonicity, growth, derivative bounds, and exact lower-Lambert identities. Separate modules for small-argument expansions later translate λ into elementary logarithms to any fixed order.

The reciprocal scale conversion is algebraically total: `smallArgumentLog_inv_eq_all` holds for every real input, and `smallArgumentLog_inv_isBigO` holds at an arbitrary filter. The small-positive filter reappears only when these identities are transferred to an endpoint asymptotic theorem.

17.3 Exact reduction at the saddle

`FabiusSharpExactReduction.lean` packages the inversion calculation into an exact error decomposition. In logarithmic form, it has the conceptual shape

$$\log F(x) - M(x) = \log\bigl(\text{normalized saddle mass}\bigr) + \text{forward-tail correction},$$

where $M(x)$ contains the explicit quadratic-logarithmic, Lambert, Gaussian, and periodic pieces.

This theorem is the critical interface between contour analysis and asymptotic algebra. All later approximations can focus on two controlled terms: a normalized central integral tending to one, and an exponentially flat tail inherited from the negative-Laplace product.

17.4 Central and tail decomposition

Write the vertical contour variable as $s = \sigma + \mathrm{i}y$. The integral is split at a central radius $|y| \le Y(\lambda)$:

$$I = I_{\text{central}} + I_{\text{tail}}.$$

Near the saddle, a Taylor expansion of the exponent begins with a negative quadratic term. Away from the saddle, a minor-arc estimate shows strict decay of the real part.

The module family includes `FabiusSaddleCentral.lean`, `FabiusSaddleTail.lean`, `FabiusLambertMinorArc.lean`, and reference-weight modules. The split allows different techniques:

- local cumulant expansions and Gaussian comparison in the center;
- coarse but uniform exponential inequalities in the tail.

Trying to use the local Taylor series on the entire contour would give an unjustified remainder.

17.5 The Gaussian reference weight

After rescaling y by the square root of the second derivative at the saddle, the leading central factor is $e^{-u^2/2}$. `FabiusSaddleReferenceWeight.lean` and `FabiusSaddleReferenceTail.lean` collect exact Gaussian integrals and tail bounds in the normalization used by the contour.

The normalized saddle mass is divided by the Gaussian main integral so its limit should be one. This normalization simplifies logarithms: once the ratio is $1 + O(1/\lambda)$, its logarithm is controlled by a standard $\log(1 + u)$ estimate.

17.6 Cumulants and derivative bounds

The logarithm of the negative-Laplace product has derivatives expressible as dyadic sums of kernel derivatives. `LaplaceCumulantAsymptotics.lean`, `FabiusLambertDerivativeBounds.lean`, and related modules prove uniform estimates for these cumulants at the saddle.

The same cumulant module packages Bell-polynomial inversion of normalized Laplace moments through order four. The first identity is already exact without side conditions:

$$\mathrm{normalizedLaplaceMoment}_1(s) = -L'(s).$$

The second derivative determines the Gaussian width. Higher derivatives determine correction polynomials. The source proves bounds of the form

$$\kappa_r(\lambda) = O(\lambda^{1-r/2})$$

after the chosen normalization. Exact exponents vary with convention, but the hierarchy guarantees that each additional Taylor order contributes another inverse power of the large phase.

Two vertical-log modules make the derivative interface explicit. For every m, `exists_norm_iteratedDeriv_negativeLaplaceVerticalLog_rpow_le_add_one` supplies $C \geq 0$ such that the $(m+1)$-st vertical derivative at radius 2^b is bounded by $C(b+1)$ for $b \geq 0$ and $|\theta| \leq 1$, including the base radius $2^0 = 1$. The legacy Cb estimate is retained for $b \geq 1$. At any radius $r > 0$, `iteratedDeriv_negativeLaplaceVerticalLog_at_zero_eq_ordinary_all` identifies every zero-axis jet: order zero is normalized to zero, while order $n > 0$ is $(r\mathrm{i})^n$ times the n-th ordinary derivative of the negative-Laplace logarithm at r.

17.7 Minor arcs and tail decay

A complex product may have oscillatory peaks away from zero. `FabiusLambertMinorArc.lean` proves that, outside the central window, enough sinc/Bose factors lose modulus to force decay. The proof divides the frequency axis into ranges:

1. a near-central range where a quadratic cosine or exponential inequality applies;
2. an intermediate range where a fixed block of dyadic factors supplies uniform loss; and
3. a far range controlled by elementary transform decay and integration by parts.

Each range has a separate numerical inequality, making the final integral estimate a finite case split rather than an opaque global bound.

17.8 The first correction

`FabiusFirstSaddleCorrection.lean` computes the first non-Gaussian term. Cubic and quartic derivatives of the phase contribute through Gaussian moments. Odd terms integrate to zero against the even Gaussian; the first surviving coefficient combines the quartic term and the square of the cubic term.

The code represents the correction as a polynomial in the rescaled central variable. Exact Gaussian moments are evaluated recursively or through double factorials. The resulting rational coefficient is then inserted into the normalized saddle mass expansion.

17.9 The corrected sharp formula

`FabiusSharpAsymptotic.lean` states the sharp logarithmic asymptotic in the repository's exact normalization. Writing Ψ for the bounded nonconstant period-one correction and C_* for the proved normalization constant, its main term is

$$\log F(x) = -\frac{\log 2}{2}\lambda(x)^2 + \left(1 + \frac{\log 2}{2}\right)\lambda(x) - \frac{1}{2}\log\lambda(x) + C_* + \Psi\big(\lambda(x)\big) + O\left(\frac{1}{\lambda(x)}\right),$$

where $\lambda(x)$ is the exact lower-Lambert coordinate. In particular, the quadratic coefficient is $-\log 2/2$; changing logarithm bases without changing the definition of λ would give a different and incorrect coefficient.

Exponentiating gives an asymptotic equivalence for $F(x)$ itself. The proof first bounds the logarithmic error, then uses continuity of the exponential and the fact that $e^{O(1/\lambda)} = 1 + O(1/\lambda)$.

17.10 Failure of an uncorrected elementary formula

A simpler formula obtained by dropping the periodic term may still reproduce the leading $(\log x)^2$ scale. It does not have a residual of order $1/(-\log x)$. The repository proves this by choosing two phase subsequences along which the periodic correction approaches different values. The residual therefore has distinct limit points and cannot tend to zero.

Similarly, a proposed quotient of elementary exponentials is shown to decay on the wrong scale. `FabiusQuotientExponentialMismatch.lean` compares logarithms and proves the quotient is not asymptotically equivalent to F, even if a finite numerical range looks persuasive.

Architectural point. The sharp theorem does not emerge from manipulating the delay equation alone. The delay equation explains the coarse quadratic scale. The periodic term and correct constant require the exact transform product; the Gaussian factor and inverse-phase corrections require Bromwich inversion and saddle analysis.

---

18 All-order asymptotic expansions

18.1 Why the exact phase is retained

An all-order expansion can be expressed either in the exact saddle variable $\lambda(x)$ or directly in powers of $-\log x$ and iterated logarithms. The repository takes the exact-phase expansion as primary. This avoids a common error: composing two finite asymptotic series and claiming an arbitrary-order remainder without proving uniform control of the composition.

The exact-phase theorem has the form

$$\mathcal{M}(x) \sim \sum_{j\geq 0} \frac{c_j(\lambda(x))}{\lambda(x)^j},$$

where $\mathcal{M}(x)$ is the normalized saddle mass and each c_j is a bounded period-one coefficient function. For every N, the difference from the first N terms is $O(\lambda^{-N})$. Replacing λ by a finite elementary-log approximation inside these oscillatory coefficients requires a separate composition theorem and is not part of this statement.

18.2 Formal logarithm of the Laplace product

`FabiusLambertFormalLog.lean` extracts a formal series for the local phase. It treats logarithms, exponentials, and reciprocal series in a truncated polynomial ring. The goal is algebraic: compute coefficients that must occur if a sufficiently smooth analytic remainder is controlled.

Separating formal algebra from analytic remainder estimates is essential. Formal power-series identities can be checked by ring normalization and coefficient extensionality. Their analytic validity is supplied later by bounds on the omitted terms.

18.3 Higher small-argument expansion

`FabiusLambertHigherExpansion.lean` and `FabiusLambertAllOrderSmallArgument.lean` prove the expansion of the exact Lambert phase in terms of

$$L = \log(1/x), \qquad \ell = \log L.$$

For each fixed order, λ is represented by a polynomial in $1/L$ whose coefficients are polynomials in ℓ, plus a rigorously bounded remainder.

The proof uses the implicit saddle equation. An approximate phase is substituted into the equation; formal algebra shows cancellation through the desired order. Monotonicity and a derivative lower bound for the implicit function turn the equation residual into a bound on the phase error.

18.4 Gaussian polynomial contractions

The central exponential has an expansion

$$e^{-u^2/2} \exp\left(\sum_{r\geq 3} \lambda^{1-r/2} a_r (iu)^r\right).$$

Truncating the second exponential produces polynomials in u and $1/\sqrt{\lambda}$. Odd monomials vanish after integration; even monomials contract to Gaussian moments. The modules `GaussianPolynomialContraction.lean`, `GaussianPolynomialWholeIntegral.lean`, and the tail modules formalize this mechanism.

A polynomial is represented by finitely supported coefficients. The contraction functional maps u^{2k} to $(2k-1)!!$ and odd powers to zero. Proving that contraction commutes with finite sums and scalar multiplication turns the central coefficient calculation into exact algebra.

18.5 Coefficient recurrences

`FabiusSaddleCoefficientRecurrence.lean`, `FabiusSaddlePolynomialCoefficients.lean`, and `FabiusSaddleExpansionCoefficients.lean` organize the coefficients recursively. Each order depends only on finitely many cumulants and lower coefficients. This recurrence remains the executable foundation even though newer modules also solve its Fabius-specific pieces in closed form.

The recurrence is justified by differentiating or multiplying truncated formal series and comparing coefficients. Lean's finite-support polynomial representation ensures every coefficient sum is finite. Index constraints are handled with `Finset.antidiagonal` or explicit bounded ranges.

`FabiusSaddleJetClosedForm.lean` solves the differential recurrence for every periodic saddle jet, while `FabiusSaddleExponentClosedForm.lean` reduces each exponent coefficient to one jet term plus a universal jet-free tail. The jet weights are the coefficients of

$$\prod_{k=1}^{n} (X-k).$$

With signed first-kind Stirling numbers defined by $X(X-1)\cdots(X-n+1)=\sum_m s(n,m)X^m$, `FabiusSaddleJetStirling.lean` identifies the weight at X^m as $s(n+1,m+1)$, not $s(n,m)$. Thus the closed form is finite and machine-checkable, while the recurrence remains the convenient generator. `FabiusSaddleLeadingCoefficient.lean` additionally isolates the top-jet coefficient of every saddle *mass* coefficient. Its header notes the analogous logarithmic transfer, but that transfer is not proved in this module.

18.6 Universal endpoint-transfer polynomials and finite expectations

Two explicitly post-snapshot modules isolate the finite algebra behind an endpoint-transfer step. `EndpointTransferPolynomials.lean` defines universal polynomials $P_r \in \mathbb{Q}[X]$ by the formal identity

$$\exp\left(-\sum_{j\geq 2}\frac{X^j t^{j-1}}{j}\right)=\sum_{r\geq 0}P_r(X)t^r.$$

Theorems `endpointTransferPolynomial_succ`, `endpointTransferPolynomial_eq_partitionExpSum`, and `endpointTransferSeries_eq_exp_subst` connect the recursive, weighted-partition, and formal-series descriptions. The evaluation theorems `aeval_endpointTransferPolynomial` and `map_endpointTransferSeries` show that the construction is natural in every commutative rational algebra. These are formal-algebra statements: they assert no analytic convergence, branch choice, or remainder estimate.

`PolynomialExpectationCumulant.lean` supplies the complementary finite integration bridge. The theorem `integral_eval`$_2$`_eq_sum_moment` expands the integral of a polynomial evaluation into coefficient-weighted raw moments, assuming integrability only for powers in the polynomial's support. The theorem `integral_eval`$_2$`_eq_sum_completeBell_momentCumulant_with_mass_correction` rewrites those moments through the complete Bell transform of their formal cumulants and adds the exact constant-term correction $\varphi(p_0)(m_0-1)$. The normalized theorem `integral_eval`$_2$`_eq_sum_completeBell_momentCumulant_of_moment_zero_eq_one` and the probability-measure specialization `integral_eval`$_2$`_eq_sum_completeBell_momentCumulant` remove that correction. Taking $p=P_r$ composes the two finite interfaces, but neither module proves the analytic endpoint-transfer expansion, Taylor remainders, or tail estimates.

18.7 Geometric principal specialization and all residual moments

At post-snapshot checkpoint `1e761ed77583e9870ceeaeb2c74ac824094f0f3f`, the finite interpolation algebra acquires a denominator-free geometric endpoint. Five reusable bridges in `CompleteHomogeneous.lean` are `completeHomogeneousEval_smul`, `completeHomogeneousEval_option_zero`, `completeHomogeneousEval_fin_succ`, `completeHomogeneousEvalOn_range`, and `completeHomogeneousEvalAt_zero`. They record homogeneity, removal of an adjoined zero variable, the head–tail recurrence on a nonempty `Fin` family, reindexing of a natural-number range by `Fin`, and specialization of the distinguished target to zero.

Over every commutative semiring, without division or a hypothesis on q, `GeometricCompleteHomogeneous.lean` proves

$$h_n(1,q,\ldots,q^p)=\begin{bmatrix}n+p\\ p\end{bmatrix}_q,\qquad h_n(c,cq,\ldots,cq^p)=c^n\begin{bmatrix}n+p\\ p\end{bmatrix}_q.$$

The exact declarations are `completeHomogeneousEval_geometric`, `completeHomogeneousEval_scaled_geometric`, and the range-indexed bridge `completeHomogeneousEvalOn_range_pow_eq_gaussianBinomial`.

The commutative-ring theorems in `GeometricResidualMoments.lean` assume polynomial exactness directly on the displayed scaled nodes:

$$\sum_{k=0}^{p} w_k(cq^k)^d = 0^d \quad (0 \le d \le p).$$

They then give, for $r \ge 0$ and $m > 0$,

$$\sum_{k=0}^{p} w_k(cq^k)^{p+1+r} = c^{p+1+r}(-1)^p q^{\binom{p+1}{2}} \begin{bmatrix} p+r \\ p \end{bmatrix}_q, \qquad \sum_{k=0}^{p} w_k(cq^k)^m = c^m(-1)^p q^{\binom{p+1}{2}} \begin{bmatrix} m-1 \\ p \end{bmatrix}_q.$$

The theorem with this direct scaled-node hypothesis is strictly more general than a result that first assumes exactness before dilation: it remains valid when c is zero or a zero divisor. The unscaled declarations are `sum_weight_mul_geometric_pow_succ_add` and `sum_weight_mul_geometric_pow_of_pos`; the maximally general scaled forms are `sum_weight_mul_scaled_geometric_pow_succ_add` and `sum_weight_mul_scaled_geometric_pow_of_pos`.

For a field, finite-node injectivity is used only to obtain those low moments from the geometric Lagrange row. The four resulting declarations are `sum_geometricLagrangeWeight_mul_pow_succ_add`, `sum_geometricLagrangeWeight_mul_pow_of_pos`, `sum_geometricLagrangeWeight_mul_scaled_geometric_pow_of_pos`, and `sum_geometricLagrangeWeight_mul_shifted_pow_of_pos`. In the final form the nodes are $q^{start+k}$, and the exponent of q is exactly $start\,m + \binom{p+1}{2}$. These sixteen declarations are finite algebra: they assert no analytic Richardson remainder, convergence rate, or sign bound for a sampled function.

18.8 Uniform central remainders

Formal coefficients alone do not prove an asymptotic expansion. `FabiusSaddleCentralAllOrders.lean` bounds the difference between the exact central integrand and its truncated polynomial approximation uniformly on the growing central window.

The window grows slowly enough that the normalized Taylor remainder tends to zero at the requested inverse-phase rate, but fast enough that the omitted Gaussian tail is negligible. Choosing this window is a balancing act. The proof records explicit inequalities among its powers of λ, so later estimates reduce to comparisons of monomials.

18.9 All-order tails

`FabiusSaddleTailAllOrders.lean`, `GaussianPolynomialTailAllOrders.lean`, and `FabiusLambertAllOrderRemainder.lean` show that tail errors are smaller than every prescribed inverse power. Exponential decay is converted into polynomial decay using standard eventual inequalities:

$$e^{-c\lambda^\alpha} = O(\lambda^{-N}) \quad \text{for every fixed } N.$$

The source proves such comparisons with explicit thresholds and reuses them through Big-O transitivity.

18.10 Assembling the normalized mass expansion

`FabiusSaddleMassAllOrders.lean` combines central coefficient extraction and tail flatness. For each N, it proves

$$\mathcal{M}(x) - \sum_{j=0}^{N-1} \frac{c_j(\lambda(x))}{\lambda(x)^j} = O(\lambda(x)^{-N}).$$

The theorem is formulated through a structure or predicate such as `HasAsymptoticExpansion`, allowing generic lemmas for truncation, addition, multiplication, and logarithms.

Two generic interfaces make the all-order statement reusable. `SaddleExpansionFlat.lean` proves that, once one full expansion is known, another function has the same coefficient family exactly when their difference is $O(\mathrm{scale}^N)$ for every N; this is the natural uniqueness statement modulo flat remainders without assuming a nondegenerate scale. `SaddleExpansionSmallArgument.lean` transfers full expansions in both directions between the exact logarithmic coordinate and $x \to 0^+$, for scales in arbitrary normed fields and functions valued in arbitrary normed vector spaces.

18.11 From mass to logarithm

Since $c_0 = 1$, the normalized mass is eventually close to one. A formal logarithm of the coefficient series gives

$$\log \mathcal{M}(x) \sim \sum_{j\geq 1} \frac{d_j(\lambda(x))}{\lambda(x)^j},$$

where the d_j are again bounded period-one functions. The analytic proof uses a Taylor theorem for $\log(1+u)$ with a uniform remainder on a small neighborhood. Positivity of the exact mass, inherited from the original integral representation, ensures the logarithm is well-defined.

At the formal level, `SaddleLogExpansionAlgebra.lean` proves that the recursive logarithm and exponential coefficient transforms are exact inverses in every commutative $\mathbb{Q}$-algebra. Unit and zero constant-term hypotheses give identities at all indices; at every positive index the inverse identities need no normalization because the recurrences do not read the zeroth input coefficient. `SaddleLogExpansionPowerSeries.lean` then identifies the recursive logarithm with Mathlib's `PowerSeries.logOf`.

`FabiusSecondSaddleCorrection.lean` computes d_2 explicitly as a finite polynomial in the first four bounded exponent jets, and `FabiusSecondSaddleExpansion.lean` substitutes it into the public three-term statement:

$$\log F(x) = \mathrm{Main}(x) + \frac{A_1(\lambda(x))}{\lambda(x)} + \frac{A_2(\lambda(x))}{\lambda(x)^2} + O\big(\lambda(x)^{-3}\big)\,.$$

Both corrections are explicit continuous period-one functions; later regularity results may provide stronger smoothness where stated. The theorem does not assert that either correction is nowhere zero.

18.12 Transfer to $F(x)$

`FabiusFullAsymptoticExpansion.lean` inserts the logarithmic mass expansion into the exact saddle reduction, adds the periodic and explicit main terms, and absorbs the exponentially small forward tail. The result is an all-order logarithmic expansion for $F(x)$ at zero.

The primary remainder is $O(\lambda^{-N})$. A weaker but simpler corollary uses $\lambda(x) \asymp -\log x$ to obtain

$$O\big((-\log x)^{-N}\big).$$

18.13 First-order and Wikipedia-facing corollaries

Modules with names such as `FabiusWikipediaMain.lean` and `FabiusWikipediaExpansion.lean` package concise formulas suitable for comparison with public summaries. They separate what is correct at leading order from what is false at the claimed next order. The obstruction module demonstrates that a nonconstant dyadic-periodic correction cannot be absorbed into a vanishing error.

18.14 The proof-audit role of the asymptotic aggregator

`PaperFabiusAsymptotic.lean` is not just a theorem list. Its header records which informal claims are supported, which need corrected domains or periodic terms, and which are disproved. The formal development therefore functions as an audit trail:

- coarse log-squared behavior is validated;
- the exact delay relation is restricted to its true domain;
- a claimed overly small residual is refuted;
- the periodic Lambert-phase formula is proved independently through transforms and saddles; and
- arbitrary-order corrections are established at the exact phase.

19 Legendre expansions and polynomial approximation

19.1 Why Legendre polynomials are a natural basis

The up-function is supported on $[-1, 1]$, is even, and is infinitely differentiable. Legendre polynomials are orthogonal on exactly this interval with respect to Lebesgue measure, so they provide a canonical spectral expansion:

$$\mathrm{up}(x) = \sum_{n\geq 0} a_n P_n(x).$$

Evenness forces every odd coefficient to vanish. The repository does not stop at a formal L^2 expansion: it evaluates the even coefficients exactly through moments, proves absolute and uniform convergence, translates the series back to the global Fabius function, and proves least-squares optimality of finite truncations.

19.2 Building the polynomial basis

`LegendrePolynomial.lean` develops the required Legendre facts in the normalization used by the project. The polynomials are defined through Rodrigues' formula or an equivalent derivative representation:

$$P_n(x) = \frac{1}{2^n n!}\frac{\mathrm{d}^n}{\mathrm{d}x^n}(x^2 - 1)^n.$$

The module proves degree, parity, endpoint values, differential equation, and derivative bounds.

A bespoke development is useful because later coefficient formulas require explicit monomial coefficients. A black-box orthogonal-polynomial theorem would provide orthogonality but not necessarily the exact finite sum compatible with the moment sequence.

19.3 Sturm–Liouville orthogonality

The Legendre differential equation is

$$\big((1 - x^2)P_n'(x)\big)' + n(n + 1)P_n(x) = 0.$$

Multiplying the equation for P_n by P_m, the equation for P_m by P_n, subtracting, and integrating gives

$$\big(n(n + 1) - m(m + 1)\big)\int_{-1}^{1} P_n(x)P_m(x)\,\mathrm{d}x = 0.$$

Boundary terms vanish because of the factor $1 - x^2$. For $m \neq n$, the eigenvalues differ, proving orthogonality.

`FabiusLegendreSeries.lean` carries out this argument using interval integration by parts. The proof explicitly verifies differentiability of the polynomial expressions and endpoint vanishing. The eigenvalue inequality is discharged in natural arithmetic and cast to reals only at the final multiplication step.

19.4 Norms and pointwise bounds

The exact norm is

$$\int_{-1}^{1} P_n(x)^2 \, \mathrm{d}x = \frac{2}{2n+1}.$$

The source derives it from Rodrigues' formula and repeated integration by parts, or from a recurrence plus normalization depending on the helper theorem used. This norm fixes the Fourier–Legendre coefficient:

$$a_n = \frac{2n+1}{2} \int_{-1}^{1} \mathrm{up}(x) P_n(x) \, \mathrm{d}x.$$

A bound $|P_n(x)| \leq 1$ on $[-1,1]$ is proved from the differential equation and an extremum argument. This simple uniform bound is the last ingredient needed to convert absolute convergence of coefficients into uniform convergence of the function series.

19.5 Vanishing of odd coefficients

The parity theorem states

$$P_n(-x) = (-1)^n P_n(x).$$

Since up is even, the integrand $\mathrm{up}(x)P_n(x)$ is odd when n is odd. A symmetric-interval integral of an odd integrable function is zero. In Lean, the proof either invokes a generic odd-integral lemma or performs the substitution $x \mapsto -x$ and adds the two integral identities.

The resulting series is indexed naturally by P_{2n}. This is not only cosmetic: the exact coefficient formula then uses the even moment M_{2k}, for which the arithmetic normalization and denominator theory are available.

19.6 Exact coefficient evaluation

`FabiusLegendreCoefficients.lean` expands P_{2n} in monomials:

$$P_{2n}(x) = \sum_{k=0}^{n} c_{n,k} x^{2k}.$$

Finite linearity of the integral gives

$$a_{2n} = \frac{4n+1}{2} \sum_{k=0}^{n} c_{n,k} M_{2k}.$$

Every M_{2k} is the exact rational `moment` value, so a_{2n} is rational. The source also substitutes formulas expressing moments through inverse-power Fabius values, producing a coefficient formula entirely in terms of the exact dyadic table.

The proof is layered:

1. establish the explicit polynomial coefficient theorem;
2. turn polynomial evaluation into a finite sum;
3. commute the finite sum with the integral;

4. replace each monomial integral by the analytic moment;
5. identify the moment with the exact rational sequence; and
6. simplify casts and normalizations.

At no point is an infinite series exchanged with the integral during coefficient evaluation.

19.7 Generic convergence infrastructure

`LegendreSeriesConvergence.lean` develops a theorem for smooth functions whose repeated derivatives satisfy suitable endpoint or norm bounds. Integration by parts transfers derivatives from the function to the Legendre eigenstructure, yielding decay of coefficients faster than any fixed power of n.

For the Fabius up-function, compact support and smoothness supply these hypotheses to every order. Consequently the coefficient sequence is absolutely summable. Since $|P_n(x)| \leq 1$, the Weierstrass M-test gives uniform convergence on $[-1, 1]$.

The generic theorem is separated from Fabius-specific coefficient arithmetic. This permits future reuse for other compactly supported smooth functions and keeps the main convergence proof independent of exact moment formulas.

19.8 Identifying the uniform limit

Orthogonality first shows that the partial sums are the L^2 projections of up. Absolute/uniform convergence produces a continuous limit S. To prove $S = \mathrm{up}$, the source may use one of two equivalent routes:

- completeness of Legendre polynomials in $L^2[-1, 1]$, followed by equality of continuous representatives; or
- a generic convergence theorem already identifying the pointwise limit for sufficiently smooth functions.

The final theorem states a `HasSum` at each point, then upgrades to uniform convergence of the function series.

Endpoints are included. This matters because pointwise convergence theorems for orthogonal series often state only interior convergence. The absolute coefficient bound and $|P_n(\pm 1)| = 1$ handle endpoints directly.

19.9 Translated series for the global Fabius function

`FabiusTranslatedLegendreSeries.lean` transports the up-function expansion to an interval such as $[0, 2]$ where a translate agrees with the signed global Fabius function. The transformation replaces $P_{2n}(x)$ by $P_{2n}(x - 1)$ or the corresponding affine normalization.

The module also expands translated Legendre polynomials into ordinary monomials. This produces an explicit power-polynomial series in the translated coordinate, with exact rational coefficients. Uniform convergence follows from the original series under composition with the affine map.

19.10 Least-squares optimality

`FabiusLegendreLeastSquares.lean` proves that the partial Legendre sum is the unique least-squares approximant among polynomials of the corresponding degree. Evenness gives a useful strengthening: the visible even sum through P_{2N} is the full orthogonal projection through degree

$2N + 1$, because the next odd coefficient vanishes. It is therefore uniquely least-squares optimal among *all* real polynomials of degree at most $2N + 1$, one degree beyond its own visible degree. Let

$$S_N^{\text{even}} = \sum_{n=0}^{N} a_{2n} P_{2n}.$$

For any polynomial p with $\deg p \leq 2N + 1$,

$$\|\mathrm{up} - p\|_2^2 = \|\mathrm{up} - S_N^{\text{even}}\|_2^2 + \|S_N^{\text{even}} - p\|_2^2.$$

This Pythagorean identity follows because $\mathrm{up} - S_N^{\text{even}}$ is orthogonal to every polynomial in that degree class.

The formal proof represents the finite span either as a submodule of L^2-type functions or directly through polynomial coefficients. It proves orthogonality term by term and then expands the square of the norm. Uniqueness follows because equality of errors forces $\|S_N^{\text{even}} - p\|_2 = 0$, hence equality almost everywhere; polynomial continuity upgrades this to pointwise polynomial equality.

19.11 Computational significance

The exact coefficient theorem gives a certified way to produce rational polynomial approximants. The least-squares theorem certifies optimality in L^2, while uniform convergence certifies pointwise approximation. This is a rare combination:

- exact arithmetic coefficients;
- kernel-checked analytic convergence; and
- an optimality theorem for every finite truncation.

The evaluator and moment recurrences can be used to compute coefficients without numerical integration.

20 The k-fold Thue–Morse draft audit

20.1 Purpose of the audit

`PaperKFoldThueMorse.lean` collects a separate line of work concerning k-fold Thue–Morse constructions and a related informal draft. Its role is both constructive and diagnostic: it proves exact identities and corrected approximation statements, but it also formalizes counterexamples to several literal claims in the draft.

This part of the repository illustrates a mature use of formalization. A failed proof is not treated as an obstacle to be papered over; the surrounding hypotheses are tested, false versions are refuted, and repaired versions are isolated.

20.2 Exact finite identities

The Thue–Morse modules prove prefix-sum cancellation, long zero runs of finite differences, convolution formulas, and generating-series identities. Repeated finite difference annihilates polynomials below a threshold degree, which explains the high-order cancellation of the sign sequence.

The full row-level symmetry is sharper than the endpoint cancellation. If $k, r, d \in \mathbb{N}$, $k \leq r$, and $k + d < 2^r$, then

$$\sigma_k(2^r - k - 1 - d) = (-1)^{r-k}\sigma_k(d), \qquad \sigma_k(2^r - k - 1 - d) = 0 \iff \sigma_k(d) = 0.$$

These are exactly `iteratedPrefix_dyadic_reverse_window` and `iteratedPrefix_dyadic_reverse_window_eq_zero_iff` in `ThueMorsePrefix.lean`. At $k = 0$, the identity is the binary complement theorem `thueMorseSign_dyadic_complement` from `DyadicClosedForm.lean`; at $d = 0$, the statement specializes, using `iteratedPrefix_at_zero`, namely $\sigma_k(0) = 1$, to the sharp left boundary `iteratedPrefix_before_dyadic_run` of the final dyadic zero run. The terminal zero run and the value -1 at its right boundary are separate theorems, exposed by `iteratedPrefix_dyadic_endpoint`, `iteratedPrefix_dyadic_zero_reverse`, `iteratedPrefix_dyadic_zero_run`, and `iteratedPrefix_at_dyadic`. In particular, the reflected-zero equivalence pairs zeros in the pre-run window but does not assert that the window is zero-free.

The proof mirrors the finite-difference geometry. Dyadic bit complementation settles order zero, the left boundary seeds offset zero, and a nested induction uses $\sigma_{k+1}(n+1) - \sigma_{k+1}(n) = \sigma_k(n+1)$ to propagate the same signed reflection in both prefix order and offset. Since the sign is always ± 1, the zero-locus equivalence follows without any cancellation hypothesis.

The formal proofs use induction on binary depth and repeated even/odd splitting. Convolution associativity is kept finite, so all sums are over `Finset.range` and can be reindexed explicitly. Generating polynomials turn convolution into multiplication, allowing coefficient comparison to replace nested sum manipulations.

`ThueMorseBinomialLog.lean` also makes the zero–one convention exact. For the integer sum $X(n) = \sum_{k=0}^{n} (-1)^{\binom{n}{k}}$, it proves $n + 1 - X(n) = 2^{\mathrm{wt}(n)+1}$, so the ensuing base-two logarithm is exact. If $t_n = \mathrm{wt}(n) \bmod 2$, the resulting rational identity is

$$t_n = \frac{1 + (-1)^{\log_2(n+1-X(n))}}{2}.$$

The module also identifies, in every ring, $(-1)^{t_n}$ with the cast of the signed Thue–Morse convention used by the real and complex q-series.

20.3 The failed polygon and the corrected grid samples

The literal grid of the draft uses $S_j^k/2^{\binom{k}{2}}$ at $j/2^k$. At $x = 1$ its value is $-2^{-\binom{k}{2}}$, so neither the grid nor either intended polygonal interpolation can converge there to a function with value one. This is formalized by `paperPrefixGridValue_endpoint_not_tendsto_one`, `paperPrefixPolygon_endpoint_not_tendsto_one`, and `paperPrefixPolygonReal_endpoint_not_tendsto_one` in `ThueMorseGenerating.lean`.

The repaired theorem uses the shifted, dyadic-floor samples `correctedPrefixGridSample`, not a continuous polygon. Away from the right endpoint each sample is exactly `stepApproximantn` at `correctedPrefixCellCenternx`; those centers tend to $2x - 1$. The endpoint is handled separately, giving pointwise convergence on $[0, 1]$ to $\mathrm{up}(2x - 1)$, and sampling at $x/2$ recovers $F(x)$. No quantitative uniform rate or continuity of the sample function is asserted.

20.4 Formal counterexamples

`DraftCounterexamples.lean` gives exact witnesses for three distinct failures. At $k = 2, j = 1$, the proposed slope is -2, whereas $F'(1/4) = 1$; the error is therefore $3 > 4h = 1$, contradicting the claimed local bound. For every $x > 0$, the proxy ratio from equation (7) is

$$\frac{x\, 2^{n+1}}{n+1} \longrightarrow +\infty,$$

so the terms are unbounded and the displayed maximum does not exist. Finally, under the draft's equation (8), the Stirling proxy retains an additive $+n$ term; equation (10) cannot absorb it into $O(\log n)$.

These are arithmetic or elementary asymptotic counterexamples, not subsequence arguments imported from the coarse Fabius endpoint theorem. Each negative result is stated independently of the prose whose claim it refutes.

20.5 Repairing the lower Lambert branch

The draft uses a lower branch of a Lambert-type inverse. The repository develops this branch with a correct domain, monotonicity, and inversion theorem. Endpoint behavior and branch separation are proved explicitly; no appeal is made to an ambiguous multivalued inverse.

These lemmas then support the corrected asymptotic comparisons in both the k-fold project and the principal Fabius saddle analysis.

20.6 Decay comparisons

`FabiusFlatness.lean` and `FabiusDecayComparison.lean` convert the log-squared law into two useful endpoint comparisons. For every $m > 0$,

$$F(x) = o(x^m) \qquad (x \downarrow 0),$$

so the function is smaller than every power. On the other hand, for every $c > 0$,

$$e^{-c/x} = o(F(x)),$$

so it is much larger than an $e^{-c/x}$ flat function.

Taking logarithms reduces these statements to comparing $(\log(1/x))^2$ with $\log(1/x)$ and with $1/x$. The code packages positivity and eventual monotonicity so the exponential comparison can use standard filter lemmas.

21 Recurring proof-engineering patterns

21.1 Strong induction as the default arithmetic engine

Many sequences in the project depend on all smaller indices, not only on the predecessor. The idiomatic proof begins with strong induction:

```
refine Nat.strong_induction_on n ?_
intro n ih
-- ih : forall m < n, P m
```

A finite sum over `Finset.rangen` then supplies the inequality needed to invoke `ih`. It is often worth proving a helper lemma that converts membership in the range directly into the strict inequality expected by the hypothesis.

Strong induction also matches the highest-bit evaluator, whose recursive remainder is smaller but not necessarily one less. Using the same well-founded order for definition and correctness proof keeps the program and theorem structurally aligned.

21.2 Finite sums and index transport

The repository moves frequently among:

- `Finset.rangen`;
- subtype indices `Finn`;
- integer intervals;

- list positions; and
- antidiagonals describing coefficient convolution.

Rather than rewriting all indices inside a large expression, the code usually proves an equivalence between finite index types and invokes a reindexing theorem. A clean bijection proof has four parts: the forward map, inverse map, membership preservation, and left/right inverse equalities.

For even/odd splits, a specialized theorem is especially valuable:

$$\sum_{j<2N} f(j) = \sum_{k<N} f(2k) + \sum_{k<N} f(2k+1).$$

Once established, it drives Thue–Morse identities, convolution recurrences, and parity counts.

21.3 Casts as explicit interfaces

Exact theorems live in $\mathbb{N}$, $\mathbb{Z}$, or $\mathbb{Q}$; analytic theorems live in $\mathbb{R}$ or $\mathbb{C}$. Lean will insert canonical casts, but it will not silently prove that a finite sum commutes with the cast or that a natural exponent has the intended target type.

The code therefore proves and reuses lemmas of the forms

$$\left(\sum_{k\in s} q_k : \mathbb{Q}\right)\uparrow^{\mathbb{R}} = \sum_{k\in s}(q_k : \mathbb{R}),$$
$$(2^n : \mathbb{Q})\uparrow^{\mathbb{R}} = (2 : \mathbb{R})^n,$$
$$\left(\binom{n}{k} : \mathbb{N}\right)\uparrow^{\mathbb{Q}} = \binom{n}{k} \text{ interpreted in } \mathbb{Q}.$$

In actual Lean syntax the upward arrows are ordinary coercions. Tactics such as `norm_cast`, `exact_mod_cast`, and `push_cast` are used after the expression is put in a form they recognize.

Lean pattern. Do not ask `norm_cast` to solve a goal containing unreduced divisions, nested sums, and conditionals. First rewrite the exact recurrence, move casts through finite sums, simplify powers, and only then invoke cast normalization. The repository's helper lemmas reflect this staged approach.

21.4 Pointwise derivatives instead of raw `deriv`

A theorem of type `HasDerivAtff'x` carries both differentiability and the derivative value. It composes under addition, multiplication, affine maps, logarithms, and exponentials. By contrast, an equality involving `derivfx` often creates a side goal proving `DifferentiableAtfx` before it can be rewritten.

Accordingly, the calculus modules first prove `HasDerivAt` identities and derive `deriv` corollaries only when a source theorem is stated in that notation. Iterated derivatives are handled through `iteratedDeriv` or `ContDiff` lemmas with explicit order parameters.

21.5 Piecewise functions and boundary glue

The bounded function, fixed-point transform, splines, and endpoint representatives are all piecewise. The source follows a stable pattern:

1. define each branch as a total function;
2. prove branch continuity or differentiability;
3. prove equality of branch values at the boundary;

4. apply a piecewise glue theorem; and
5. derive simplified branch lemmas for later rewriting.

The simplified lemmas are often tagged for the simplifier only when their conditions are syntactically obvious. Overly aggressive `simp` tags on overlapping branches can create loops or choose an unintended normal form.

21.6 Local finiteness before infinite-sum algebra

For translate sums, the code first proves finite support at a point or on a neighborhood. Only then does it reindex or differentiate a `tsum`. This is safer than invoking general unconditional summability theorems on a series whose convergence is really trivial by support.

A typical theorem states that the summand is zero outside a finite integer interval. From this it derives summability, rewrites the `tsum` as a `Finset.sum`, performs the algebra, and converts back only if the public theorem uses `tsum` notation.

21.7 Integrability from support and bounds

The most common integrability proof has three ingredients:

- continuity or measurability of the integrand;
- compact support; and
- a uniform bound on that support.

The repository builds reusable support lemmas for F, up, their derivatives, and affine transforms. Polynomial or exponential multipliers remain bounded on a compact support, so moment and transform integrability follows from a generic compact-support theorem.

This style avoids global estimates that are stronger than needed. For instance, $x^n\,\mathrm{up}(x)$ is integrable not because x^n is globally bounded, but because the product vanishes outside $[-1, 1]$.

21.8 Interchanging sums, limits, derivatives, and integrals

Every interchange in the project is supported by a named convergence theorem:

- finite sums commute with everything by algebra;
- infinite sums commute with integrals under a summable integrable majorant;
- limits pass under integrals by dominated or bounded convergence;
- derivatives pass through series under local uniform convergence of derivative series; and
- contour limits use explicit uniform tail bounds.

The proof normally constructs the majorant first, proves its summability or integrability, and only then invokes the interchange theorem. This order makes failures local: if an informal series is not uniformly controlled, the missing hypothesis appears before the main identity is attempted.

21.9 Big-O, little-o, and eventual domains

Asymptotic modules use filters rather than epsilon notation. A theorem of the form

$$f(x) = O(g(x)) \quad (x \to 0^+)$$

is represented through `IsBigO` at a within-neighborhood filter. The proof often begins with

```
filter_upwards [eventually_pos, eventually_small] with x hxpos hxsmall
```

and establishes a pointwise norm inequality.

Eventual positivity is particularly important for logarithms, divisions, and Lambert phases. The source proves threshold lemmas once and then includes them in filter-upwards blocks. Transitivity of Big-O and equivalence under asymptotically equal scales are used heavily to transfer results from the exact phase $\lambda(x)$ to $-\log x$.

21.10 Polynomial identities

Exact finite identities are proved at the polynomial level whenever possible. The usual techniques are:

- extensionality by coefficients;
- evaluation at an arbitrary point followed by `ring`;
- induction using polynomial addition and multiplication; and
- degree bounds plus a roots theorem.

For translation-invariant Thue–Morse sums, a degree bound and finite-difference theorem often give a cleaner proof than expanding every binomial coefficient.

21.11 Choosing automation deliberately

The development uses automation, but proofs are structured so each tactic sees an appropriate goal:

`simp`
: unfolds branch lemmas, finite-range membership, and standard algebraic identities.

`norm_num`
: closes concrete rational and natural computations, especially counterexamples and base cases.

`ring` / `ring_nf`
: handles polynomial identities after divisions and casts have been normalized.

`linarith` / `nlinarith`
: combines real inequalities once transcendental terms have been abstracted as variables with known bounds.

`positivity`
: proves positivity of products, powers, factorials, exponentials, and denominators.

`omega`
: handles Presburger arithmetic for indices, ranges, and exponents.

`field_simp`
: clears rational or field denominators after nonzero hypotheses are established.

`aesop`
: resolves local structural goals, but is not used as a substitute for choosing the main induction or analytic theorem.

21.12 Public wrappers and internal strength

Theorem-producing modules tend to prove results in a strong reusable form: unrestricted numerators, arbitrary candidate functions, general coefficient rings, or explicit error constants. Paper-facing wrappers then add notation and specialize hypotheses.

When extending the library, search for the strongest internal theorem before reproving a source-shaped special case. Names ending in `_all`, `_global`, `_scalar`, `_correct`, `_transfer`, or `_of_isFabius` often signal the reusable layer.

21.13 No placeholders and explicit corrections

The repository states that the development builds without `sorry`. More importantly, corrected claims are not represented by axioms that merely assume the missing fact. When a source formula is false, the project either adds the necessary hypothesis, changes the object to the signed extension, weakens the error term, or proves a counterexample.

This makes the dependency graph epistemically meaningful. A theorem about the Fabius function ultimately reduces to mathlib, Lean's kernel, explicit definitions, and proved lemmas—not to a paper citation embedded as an axiom.

22 How to read and extend the code

22.1 Four recommended reading paths

Different goals call for different entry points.

22.1.1 Path A: existence and uniqueness

Read, in order:

1. `Basic.lean`;
2. `Differential.lean`;
3. `DyadicClosedForm.lean`;
4. `DyadicAnalytic.lean`; and
5. `Existence.lean`.

This path explains the abstract specification, generic derivative consequences, dyadic rigidity, and fixed-point implementation.

22.1.2 Path B: exact evaluation

Read:

1. `Arithmetic.lean`;
2. `DyadicCorrectness.lean`;
3. `DyadicClosedForm.lean`;
4. `DyadicAnalytic.lean`; and
5. `GlobalDyadic.lean`.

Keep an editor evaluation buffer open and inspect the types of table, Horner, and wrapper correctness theorems.

22.1.3 Path C: the historical Rvachev theory

Read:

1. `GlobalExtension.lean`;
2. `FourierProduct.lean`;
3. `FourierAnalytic.lean`;
4. `OriginalUniqueness.lean`;

5. `ProbabilityRepresentation.lean`; and
6. `StepApproximationLimit.lean`.

This path emphasizes compact support, transform refinement, convolutions, and pointwise approximants.

22.1.4 Path D: sharp asymptotics

Read:

1. `FabiusLogScale.lean`;
2. `FabiusLogSquaredAsymptotic.lean`;
3. `NegativeLaplace.lean`;
4. `PeriodicCorrection.lean`;
5. `MellinBose.lean`;
6. `FabiusLambertPhase.lean`;
7. `BromwichSaddle.lean`;
8. `FabiusSharpExactReduction.lean`;
9. `FabiusSharpAsymptotic.lean`; and
10. `FabiusFullAsymptoticExpansion.lean`.

Use the specialized central/tail and all-order modules as dependencies are encountered.

22.2 Adding a new exact identity

Suppose a new conjectured formula concerns $F(a/2^n)$. A productive workflow is:

1. state and test the identity entirely in $\mathbb{Q}$ using `fabiusDyadicValue`;
2. normalize representations and determine whether the bounded or global extension is intended;
3. prove the finite arithmetic identity, preferably through existing closed-form or q-binomial lemmas;
4. only then transport it to the analytic function using dyadic correctness; and
5. add a paper-facing wrapper if the notation comes from an external source.

This approach keeps exploratory computation cheap and avoids starting with real analysis when the content is combinatorial.

22.3 Adding a new moment theorem

For a new moment recurrence:

1. derive the analytic recurrence in a generic `IsFabius` context;
2. define an exact rational sequence only if computation or arithmetic consequences justify it;
3. prove equality by strong induction;
4. design a division-free numerator recurrence before attempting parity or divisibility; and
5. separate reduced-denominator conclusions into a later module.

The existing normalized moment files are templates for common-denominator induction.

22.4 Adding an asymptotic correction

A proposed correction term should be attached to the exact saddle reduction rather than inferred from numerical fitting. The robust route is:

1. identify which cumulant or periodic Fourier coefficient contributes at the desired order;
2. extend the formal coefficient recurrence;
3. prove a uniform central remainder at that order;
4. prove the tail is smaller than the target scale;
5. update the normalized mass expansion; and
6. transfer through logarithm and the exact phase.

If the desired final formula is in elementary logarithms, derive it as a corollary of the exact-phase theorem using `FabiusLambertAllOrderSmallArgument.lean`.

22.5 Testing a suspected source error

Formalization is most efficient when the claim is stress-tested early.

- Check endpoint and smallest-index cases by exact evaluation.
- Verify domains of every differentiated functional equation.
- Compare bounded and signed-global versions under translation.
- For asymptotic claims, sample subsequences of fixed dyadic phase.
- For uniform errors, examine knots and support boundaries.
- For q-dependent finite rows, test nondyadic locations before claiming parameter independence.

A small counterexample theorem is often more informative than a long stalled proof attempt.

22.6 Maintaining dependency hygiene

Before importing the umbrella module into a new file, ask which interface is actually needed. An exact lemma should normally depend on `Arithmetic.lean` or a narrow q-binomial module, not on `Existence.lean`. A generic analytic lemma should depend on `Basic.lean` and the relevant calculus modules, not on the canonical instance unless the theorem truly uses it.

A useful check is to run the import-graph tooling included through mathlib's dependencies. Unexpected paths from asymptotic modules into paper aggregators or from arithmetic modules into analysis are signs that a helper theorem belongs in a lower layer.

22.7 Naming and theorem granularity

The repository's long module names encode the proof stage: `Raw`, `Scalar`, `ExactReduction`, `Transfer`, `Central`, `Tail`, `AllOrders`, and so on. New declarations should preserve this vocabulary where possible.

A theorem should expose one conceptual transition. Good boundaries include:

- closed formula equals Horner program;
- analytic recurrence equals exact recurrence;
- central integral equals Gaussian main term plus error;
- exact phase equals truncated elementary expansion plus error; and

- source statement follows from the stronger internal theorem.

Large proofs become navigable when these transitions have names.

22.8 A minimal downstream example

A downstream file that only needs exact evaluation and the analytic correctness theorem can begin as follows:

```
import FabiusFunction.PaperStatements

open Fabius

example (n : Nat) (a : Int) :
    (fabiusDyadicValue n a : Real) =
      fabiusReal fabius ((a : Real) / (2 : Real) ^ n) := by
  exact fabiusDyadicValue_cast fabius fabius_spec n a
```

This statement accepts every signed numerator, so no reduced-fraction or unit-interval side condition remains. Importing `FabiusFunction.lean` would also work, but would hide which layer supplies the public wrapper.

22.9 What to trust when prose and code differ

The kernel-checked theorem type is the final authority. Module headers and README tables explain intent and source correspondence, while the proof term certifies the exact statement. When a paper formula differs from a Lean wrapper, inspect:

1. whether the paper silently assumes an index is positive;
2. whether a removable singularity has been totalized;
3. whether a global extension replaces a bounded function;
4. whether endpoint representatives differ only almost everywhere; and
5. whether an asymptotic periodic term has been omitted.

The repository usually records the reason in the aggregator header or a nearby correction theorem.

23 Concluding perspective

The ProveIt development is best understood not as a very long proof of a single theorem, but as a small mathematical library centered on one function. It has several independent foundations:

- an exact arithmetic language of moments, binary signs, products, and dyadic evaluators;
- an abstract analytic interface capturing the Fabius differential and symmetry laws;
- a Banach fixed-point construction of the bounded solution;
- a compact-support/Fourier theory of the Rvachev twin; and
- a transform-and-saddle theory for the flat endpoint.

The strongest parts of the architecture are the bridges. The dyadic bridge identifies executable rationals with analytic values. The moment bridge identifies finite recurrences with integrals. The Fourier bridge identifies a refinement equation with an infinite product. The saddle bridge identifies that product with a corrected sharp asymptotic. The Legendre bridge turns exact moments into optimal polynomial approximants.

Formalization also changes the mathematical product. It forces endpoint conventions, domains of delay equations, transform normalizations, and bounded-versus-global distinctions to be stated. It exposes false residual estimates and missing terms, while often supplying a corrected theorem stronger than the source statement. The result is not merely a verification of known facts: it is a coherent, reusable account of how the facts fit together.

For a reader entering the source, the single most useful habit is to ask which side of the project a declaration belongs to: exact arithmetic, generic analysis, canonical construction, or a bridge. Once that question is answered, the long file list resolves into a small number of repeated proof patterns and a remarkably systematic dependency graph.

A Detailed dependency map

A.1 Core spine

The shortest conceptual spine from definitions to the canonical function is:

$$\boxed{\texttt{Arithmetic.lean}} \longrightarrow \boxed{\texttt{Basic.lean}} \longrightarrow \boxed{\texttt{Differential.lean}}$$
$$\longrightarrow \boxed{\texttt{DyadicAnalytic.lean}} \longrightarrow \boxed{\texttt{Existence.lean}}$$
$$\longrightarrow \boxed{\texttt{PaperStatements.lean}}.$$

This display omits two important side inputs. `DyadicClosedForm.lean` supplies the exact finite polynomial matched by `DyadicAnalytic.lean`, while `DyadicCorrectness.lean` supplies evaluator correctness and representation invariance. `Existence.lean` proves unique existence; `PaperStatements.lean` makes the canonical choice and publishes `fabius_spec`. The resulting canonical function is then consumed by moment, Fourier, probability, global-extension, and asymptotic modules.

A.2 Arithmetic spine

The main arithmetic chain is:

$$\texttt{Arithmetic.lean} \longrightarrow \left\{ \begin{array}{c} \texttt{NormalizedMoments.lean} \\ \texttt{NormalizedEvenMoments.lean} \\ \texttt{Parity.lean} \end{array} \right.$$

$$\longrightarrow \left\{ \begin{array}{c} \texttt{HalfMomentDenominator.lean} \\ \texttt{TwoAdic.lean} \\ \texttt{DenominatorBound.lean} \end{array} \right. \longrightarrow \texttt{PaperStatements.lean}.$$

`AnalyticMoments.lean` and `ExactInversePower.lean` connect this chain to the canonical function before the source-facing wrappers are assembled.

A.3 Original-paper spine

The original Rvachev paper is represented by two parallel paths:

$$\text{differential/refinement law} \to \text{Fourier recurrence} \to \text{sinc product} \to \text{uniqueness},$$
$$\text{finite elementary laws} \to \text{convolutions} \to \text{weak convergence} \to \text{step approximants}.$$

Their principal modules are `OriginalUniqueness.lean`, `FourierProduct.lean`, `ProbabilityRepresentation.lean`, `WeakConvergence.lean`, and `StepApproximationLimit.lean`. `OriginalPaperSupplement.lean` adds the remaining numbered theorems and prose claims.

A.4 Asymptotic spine

The sharp endpoint proof has the following high-level flow:

$$
\begin{gathered}
\texttt{AnalyticMoments.lean} \to \texttt{NegativeLaplace.lean} \to \left\{ \begin{array}{c} \texttt{PeriodicCorrection.lean} \\ \texttt{MellinBose.lean} \end{array} \right. \\
\to \texttt{BromwichSaddle.lean} \to \texttt{FabiusLambertPhase.lean} \\
\to \left\{ \begin{array}{c} \text{central modules} \\ \text{tail modules} \end{array} \right. \to \texttt{FabiusSharpExactReduction.lean} \\
\to \texttt{FabiusSharpAsymptotic.lean} \to \texttt{FabiusFullAsymptoticExpansion.lean}.
\end{gathered}
$$

The actual source graph is finer because the all-order proof separates formal coefficient algebra, small-argument Lambert expansion, Gaussian contraction, central uniformity, tail flatness, and transfer lemmas.

B Paper theorem map

B.1 Arias de Reyna, arXiv:1702.05442

Source item	Lean declaration or family	Principal proof location
Theorem 1: structure and fold	`IsOriginalFabius`; `IsOriginalFabius.mk_of_derivative_law`; `IsFabius.isOriginalFabius_rvachevUp`	`OriginalCharacterization.lean`
Theorem 1: uniqueness and exact fold kernel	`isOriginalFabius_iff_eq_canonical`; `rvachevUp_eq_iff_eqOn_Iic_one`; `isFabius_iff_isOriginalFabius_rvachevUp_and_rightTail`; `isOriginalFabius_iff_existsUnique_isFabius`	`OriginalUniqueness.lean`
Lemma 1 and limiting measure	`finiteConvolutionProbability_tendsto`, bounded-continuous integral corollaries, and `support_rvachevMeasure`	`EarlyApproximants.lean` (finite convolution measures), `WeakConvergence.lean` (characteristic functions, Lévy convergence, and exact support)
Theorem 2	`stepApproximant_tendsto_rvachevUp` and endpoint-corrected variants	`StepApproximationLimit.lean`
Theorem 3	probability representation of the up-function	`ProbabilityRepresentation.lean`
Theorem 4	`original_theorem_four_a`, `original_theorem_four_b`, `original_theorem_four_c`	`OriginalPaperSupplement.lean`, Fourier and differential helpers
Nonanalyticity	`rvachev_not_analyticAt`	`OriginalPaperSupplement.lean`, strengthened to the exact locus in `NowhereAnalytic.lean`

Source item	Lean declaration or family	Principal proof location
Theorem 5	Poisson summation, real-axis Fourier rapid decay, `rvachev_poisson_support_specialization_unscaled_of_one_half_le`, and `rvachev_poisson_support_specialization_of_one_half_le`	`PoissonSummation.lean`
Theorem 6	`original_theorem_six`; moment formulas	`AnalyticMoments.lean`, `OriginalPaperSupplement.lean`
Theorem 7	`original_theorem_seven_global`; global rationality/dyadic statement	`GlobalDyadic.lean`, `OriginalPaperSupplement.lean`

B.2 Arias de Reyna, arXiv:1702.06487

Source item	Lean declaration or family	Principal proof location
Proposition 1	`proposition_one`	`PaperStatements.lean`; exact moment foundations
Proposition 2	`proposition_two`; `proposition_two_real`	`AnalyticMoments.lean`, `PaperStatements.lean`; complex and real removable-singularity forms
Proposition 3	`proposition_three`	`MomentPowerSeries.lean`; even-binomial transform of half moments
Proposition 4	`proposition_four`	`NormalizedMoments.lean`; natural normalization of half moments
Question 5	`reshetnikovQuestion`	recorded as a predicate/question in `PaperStatements.lean`
Proposition 6	`proposition_six`	exact arithmetic and dyadic modules
Theorem 7	`theorem_seven`	`DyadicClosedForm.lean`, `DyadicAnalytic.lean`
Proposition 8	`proposition_eight`	`PaperStatements.lean`; even-index Reshetnikov formula and power-of-two denominator bound
Theorem 9	`theorem_nine`, strengthened all-parameter form	`Paper06487Supplement.lean`, global/differential helpers
Lemma 1	`lemma_one`, with corrected order condition	`TaylorReduction.lean`, `PaperStatements.lean`
Proposition 10	`proposition_ten`	`TaylorReduction.lean`; recursive signed-extension reduction
Definition 12	`dyadicDenominator`	`Arithmetic.lean`
Theorem 13	`theorem_thirteen` and denominator bound	`DenominatorBound.lean`
Proposition 15	`proposition_fifteen`	`PaperStatements.lean`; inverse-power denominator divides the level dyadic denominator

Source item	Lean declaration or family	Principal proof location
Conjecture 16	`conjecture_sixteen`	recorded without asserting a proof
Theorem 17	`theorem_seventeen`	`PaperStatements.lean`; Lucas digit-product congruence
Proposition 18	`proposition_eighteen`	`Parity.lean`, `PaperStatements.lean`; count of odd binomial coefficients
Proposition 19	`proposition_nineteen`	`Parity.lean`; oddness of `momentNumerator`
Theorem 20	`theorem_twenty`	`TwoAdic.lean`; odd reduced numerator/denominator and valuation of $2d_n$; corrected proof-internal product $2d_kN_n$ in `Paper06487Supplement.lean`
Theorem 21	`theorem_twenty_one`	`TwoAdic.lean`; oddness of R_n and exact inverse-dyadic two-adic valuation
Proposition 22	`proposition_twenty_two_initial`; `proposition_twenty_two`	`BernoulliRecurrences.lean`, `PaperStatements.lean`

B.3 How to use the map

The declaration named after a source theorem is normally the best citation target in downstream formal work. The principal proof location is the best learning target. When the two differ, the wrapper is preserving source notation while the internal file proves a stronger or more reusable theorem.

C Mathematical object–Lean crosswalk

Mathematical object	Principal Lean name	Defining or identifying module
Abstract bounded Fabius specification	`IsFabius`	`Basic.lean`
Canonical bounded function F	`fabius`, `fabius_spec`	chosen in `PaperStatements.lean`; existence and uniqueness proved in `Existence.lean`
Rvachev up-function up	`rvachevUp`	`Basic.lean`; analytic properties in `Differential.lean` and later shape modules
Signed global extension $\widetilde{F}$	`extendedFabius`	defined in `Basic.lean`; cell and smoothness theory in `GlobalExtension.lean`
Exact even moment c_n	`moment`, `momentIntegral`	`Arithmetic.lean`, `Basic.lean`, and `AnalyticMoments.lean`
Normalized half moment d_n	`halfMoment`, `halfMomentIntegral`	`Arithmetic.lean`, `Basic.lean`, and `AnalyticMoments.lean`

Mathematical object	Principal Lean name	Defining or identifying module
Canonical moment numerators F_n, G_n	momentNumerator, halfMomentNumerator	Arithmetic.lean, NormalizedEvenMoments.lean, and NormalizedMoments.lean
Inverse-power table $F(2^{-n})$	fabiusInversePowTwoTable, halfMomentFabiusValue	Arithmetic.lean and ExactInversePower.lean
Total dyadic evaluator	fabiusDyadicValue	Arithmetic.lean; analytic correctness wrapper in PaperStatements.lean
Dyadic common denominator D_n	dyadicDenominator	Arithmetic.lean and DenominatorBound.lean
Reshetnikov quantity R_n	reshetnikov	Arithmetic.lean and PaperStatements.lean
Weighted-sum CDF	weightedSumCDF	ProbabilityRepresentation.lean
Forward curvature profile	deriv_deriv_fabiusReal, deriv_deriv_fabiusReal_eq_zero_iff	Convexity.lean
Totalized inverse F^{-1}	fabiusInv, fabiusOrderIso, fabiusInv_differentiableAt_iff, fabiusInv_contDiffAt_infty_iff	FabiusInverse.lean
Inverse curvature	deriv_deriv_fabiusInv, deriv_deriv_fabiusInv_half, deriv_deriv_fabiusInv_eq_zero_iff, strictConcaveOn_fabiusInv_firstHalf, strictConvexOn_fabiusInv_secondHalf	FabiusInverse.lean
Uniform spline S_p	fabiusUniformSpline	FabiusUniformSpline.lean and FabiusComputability.lean
Logarithmic profile	fabiusLogProfile	FabiusLogScale.lean
Lower-Lambert reference coordinate λ	fabiusLambertPhase	FabiusLambertSaddle.lean
Periodic correction Ψ	negativeLaplacePsi	PeriodicCorrection.lean
Even Legendre truncation	rvachevLegendrePartialSumPolynomial	FabiusLegendreLeastSquares.lean

The table maps notation to declarations; it does not assert that every informal formula in either source paper is a proved theorem. In particular, reshetnikovQuestion and conjecture_sixteen record open assertions, the lower-Lambert coordinate is not claimed to solve the full residual phase's exact critical equation, the periodic correction is not constant, finite complex-q approximants need not be translation independent, and the literal k-fold polygonal limit is formally refuted.

D Annotated module guide

The following guide groups the principal files by role. The development contains many fine-grained helper modules; the descriptions emphasize the transition each module contributes rather than every local lemma.

Module	Role in the development
Core interfaces and construction	
`Arithmetic.lean`	Exact natural/rational sequences, binary weight, Thue–Morse signs, product normalizations, inverse-power values, and executable dyadic evaluators.
`Basic.lean`	The unit-interval codomain, bounded-function type, abstract IsFabius specification, up-function, signed extension names, transforms, moments, and generating functions.
`Differential.lean`	Derivative identities for any IsFabius candidate, including iterated scale formulas and reflected branches.
`Existence.lean`	Banach fixed-point construction of `Existence.boundedCandidate`, calculus bootstrap, unique existence, and generic equality of all `IsFabius` candidates. The canonical choice itself is made in `PaperStatements.lean`.
`ScaleTranslation.lean`	Reusable chain-rule formulas for affine rescaling and translation of Fabius and up-functions.
`GlobalExtension.lean`	Local finiteness, smoothness, unit-interval agreement, and translation laws of the signed global extension.
`GlobalDyadic.lean`	Exact dyadic identities for the signed extension with unrestricted and nonreduced representatives.
Global shape, inversion, and computability	
`ScalingRecurrence.lean`	Mathlib-only chain rules for arbitrary positive dilations, in vector and scalar forms, with caller-normalized outgoing weight; inert-parameter variants and dyadic wrappers used by negative-Laplace derivatives.
`Monotonicity.lean`	Strict increase of F on $[0, 1]$, endpoint range, and order consequences.
`Regularity.lean`	Optimal global Lipschitz constants for F and up.
`Convexity.lean`	Exact second-derivative formula and sign/zero loci, convexity before the midpoint, and concavity afterward.
`GlobalBounds.lean`	Sharp value and iterated-derivative bounds for the signed global extension and up, including attainment and derivative-support control.
`BoundedDerivatives.lean`	Exact global sup-norm bounds for every derivative, including attainment.
`EffectiveFlatness.lean`	Explicit power bounds at the flat endpoint.
`SharpFlatness.lean`	Factorial-strengthened endpoint power bounds for F and up.
`NowhereAnalytic.lean`	Exact real-analytic loci: off the closed transition supports and nowhere on them.
`FabiusInverse.lean`	Totalized inverse, order isomorphism on the unit interval, exact positive reciprocal derivative, exact global smoothness locus (continuity everywhere, but differentiability and every positive finite or C^∞ order exactly off the clamping endpoints), reciprocal-cubic second derivative, midpoint jet and interior curvature loci, strict closed-half curvature, and endpoint quotient/derivative blow-up.

Module	Role in the development
`InverseBranch.lean`	Analytic inverse theorem, analytic-density preservation for continuous right-inverse branches, and the complement-interior analyticity condition and conditional Lambert-branch criterion for `IsElementaryOrInverse`.
`AlgebraicBranch.lean`	Algebraic-function branch interface used in non-elementarity arguments.
`ElementaryFunction.lean`	Formal expression language and semantics for elementary real functions.
`NotElementary.lean`	Non-elementarity theorem for the bounded Fabius function on its transition interval.
`InverseNotElementary.lean`	Interior noncriticality derived from the positive derivative API in `FabiusInverse.lean`, the exact analytic locus of `fabiusInv`, and local analytic-density/non-elementarity obstructions.
`FabiusComputability.lean`	Global uniform spline error $\leq 2^{-p}$, moving-input stability and diagonal convergence, an effective uniform-continuity modulus, and the computable-real-function bridge.
`FabiusComputableSpline.lean`	Primitive-recursive bounded and signed-global dyadic spline evaluators, each with an explicit error certificate, and their computable-real interfaces.
Dyadic formulas and verified computation	
`DyadicClosedForm.lean`	Closed finite Taylor formulas, highest-bit decompositions, and equivalence among exact dyadic expressions.
`DyadicCorrectness.lean`	Termination-facing invariants, table-prefix stability, Horner correctness, refinement invariance, and evaluator representation independence.
`DyadicAnalytic.lean`	Finite Taylor recurrence for an abstract IsFabius function and proof that analytic dyadic values equal the exact evaluator.
`ExactInversePower.lean`	Links inverse-power Fabius values with exact moments and table entries.
`TaylorReduction.lean`	Finite Taylor-reduction identities used to move between dyadic cells.
`FabiusDyadicLogBounds.lean`	Explicit logarithmic bounds for inverse-dyadic values, used in endpoint asymptotics.
`DyadicSharpConditional.lean`	Conditional sharp statements at dyadic points under specified recurrence estimates.
`DyadicSharpDecomposition.lean`	Exact decomposition of dyadic logarithms into main and correction pieces.
`FabiusDyadicSharpCumulant.lean`	Cumulant organization specialized to inverse-dyadic asymptotics.
`FabiusDyadicSharpAsymptotic.lean`	Sharp asymptotic statements along the inverse-power sequence.
Moments, arithmetic, and denominators	
`AnalyticMoments.lean`	Compact-support integration, total mass, moment and half-moment recurrences, and identification with exact rational sequences.

Module	Role in the development
`MomentPowerSeries.lean`	Exact `PowerSeries` algebra over $\mathbb{Q}$, including $M(4X) = M(X)S(X)$; analytic evaluation is deferred to later modules.
`NormalizedMoments.lean`	Division-free normalization theorem for half moments using factorial and Mersenne products.
`NormalizedEvenMoments.lean`	Natural numerator and common-denominator formula for even centered moments.
`Parity.lean`	Lucas-theorem parity analysis and oddness of natural moment numerators.
`TwoAdic.lean`	Reduced numerator/denominator parity and exact two-adic valuation consequences.
`DenominatorBound.lean`	Common-denominator divisibility and explicit denominator bounds for source-facing arithmetic results.
`HalfMomentDenominator.lean`	Denominator control specialized to half moments and inverse-power values.
`BernoulliRecurrences.lean`	Translation of moment recurrences into Bernoulli-number identities used by the arithmetic paper.
`LaplaceMoments.lean`	Moment identities in the Laplace-transform normalization.
`LaplaceMomentBounds.lean`	Uniform factorial/exponential bounds needed for transform series and differentiation.
`ProbabilityLaplaceMoments.lean`	Moment and Laplace identities connecting the weighted-sum probability model with the analytic generating function.
`FabiusRecurrenceSequence.lean`	Auxiliary exact recurrence sequence used in asymptotic and dyadic estimates.
Fourier, probability, and the first paper	
`FourierProduct.lean`	Entire sinc, pointwise multipliability, finite-product recurrence, and equality of the product with the analytic Fourier transform.
`FourierAnalytic.lean`	Entire Fourier transform of up, Fourier inversion consequences, and its imaginary-axis relation to the half-moment generating function.
`OriginalCharacterization.lean`	Rvachev's original compact-support predicate, derived scale positivity, the forced scale and normalization, and the general fold from bounded Fabius solutions.
`OriginalUniqueness.lean`	Fourier-refinement uniqueness, the exact kernel of the fold, and the equivalences with fixed candidates or unique bounded Fabius witnesses.
`ProbabilityRepresentation.lean`	Product measure on $[0,1]^{\mathbb{N}}$, weighted coordinate sum, self-similarity, and exact CDF/up identifications.
`WeakConvergence.lean`	Characteristic-function convergence, Lévy continuity, bounded-continuous test convergence, positivity on every open set meeting $(-1,1)$, and exact limiting-measure support $[-1,1]$.
`StepMeasureBridge.lean`	Relates discrete/step approximants to their associated finite measures.
`EarlyMeasureBridge.lean`	Foundational bridge between early convolution approximants and measure formulations.
`StepApproximationLimit.lean`	Coefficient monotonicity, endpoint-corrected step functions, and pointwise convergence by an interval-average squeeze.

Module	Role in the development
`EarlyApproximants.lean`	Definitions and elementary properties of the finite approximants used before the limiting theorem.
`PoissonSummation.lean`	Poisson summation, rapid decay of every real-axis Fourier derivative, and upper-bound-free support specializations for $a \geq \frac{1}{2}$.
`OriginalPaperSupplement.lean`	Remaining numbered and prose results of the first paper, including exact Taylor formulas and nonanalyticity.
`Paper05442.lean`	Aggregator and coverage audit for arXiv:1702.05442.
The arithmetic paper	
`PaperStatements.lean`	Source-numbered propositions, theorems, question, definition, and conjecture for arXiv:1702.06487.
`Paper06487Supplement.lean`	Prose-level and strengthened arithmetic-paper consequences, with corrected hypotheses and bounded/global distinctions.
`Paper06487.lean`	Compact aggregator for the complete arithmetic-paper formalization.
Finite q-binomial and Thue–Morse algebra	
`HalfQBinomial.lean`	Exact q-binomial coefficients at q=1/2, q-Pascal recurrence, Mersenne normalization, and a finite q-binomial theorem.
`FabiusRawQBinomialFormula.lean`	Raw coefficient-valued finite q-binomial identity before scalar or analytic specialization.
`FabiusRawQBinomialScalar.lean`	Scalar evaluation of the raw finite identity.
`FabiusQBinomialFormula.lean`	Principal exact Thue–Morse/q-binomial formula for unrestricted natural parameters.
`FabiusQBinomialScalarFormula.lean`	Scalar-valued form of the principal q-binomial theorem.
`FabiusDyadicQBinomialScalar.lean`	Specialization using exact dyadic Fabius values.
`FabiusQBinomialTaylor.lean`	Constancy in the common translation indeterminate and identification, after normalization, with `fabiusReductionSum`.
`FabiusGlobalQBinomialSeries.lean`	Real and complex all-real q-series for the signed extension, with pointwise q-independent summands and zero-series behavior on the nonpositive side.
`FabiusBinaryReductionSeries.lean`	Exact binary telescope, the scale-zero term distinguishing even and odd unit blocks, a global 2^{1-N} residual bound, uniform convergence on $\mathbb{R}$, and all-real sum identities.
`FabiusParityPowerSeries.lean`	All-real parity-power form of the signed extension, including the integer-valued scale-zero exponent, termwise vanishing on the nonpositive side, and the restricted older $m = 1$ form.
Discrete-limit family	
`FabiusUniformSpline.lean`	Centered finite Thue–Morse splines; empty-prefix support, block translation, finite-CDF identification, monotonicity/range bounds, and pointwise convergence.
`FabiusComplexShiftSpline.lean`	Taylor control of complex shifts of finite splines.

Module	Role in the development
`FabiusDiscreteLimitToeplitz.lean`	Toeplitz-type averaging principle for normalized finite rows.
`FabiusDiscreteLimitComplexShift.lean`	Vanishing of complex-shift corrections under the limit scaling.
`FabiusDiscreteLimitIntegration.lean`	Exact outer q-binomial reindexing, Toeplitz passage for complex-shift splines, identification with the binary/literal global series, and all-real extension by empty-prefix vanishing.
Coarse endpoint asymptotics	
`FabiusLogScale.lean`	Defines the dyadic logarithmic coordinate and proves the domain-correct delay identity for the logarithm.
`FabiusLogSquaredAsymptotic.lean`	Quadratic logarithmic leading term, floor/ceiling transfer to real scale, and an explicit O(t log t) remainder.
`FabiusSmallArgumentScale.lean`	Filter and scale conversions between x tending to zero and the large logarithmic variable.
`FabiusFlatness.lean`	Proof that the Fabius function is smaller than every positive power at zero.
`FabiusDecayComparison.lean`	Comparison with exponential-flat functions such as exp(-c/x).
`FabiusLogMainDefect.lean`	Definition and first bounds for the defect after subtracting the quadratic logarithmic main term.
Negative Laplace, Mellin, and periodicity	
`NegativeLaplace.lean`	Infinite product and absolutely convergent logarithm, exact dilation equation, quadratic decomposition, periodic correction, exponential tail, and strict signs of the factors, logarithm, forward tail, and positive tail error.
`NegativeLaplaceVerticalAllOrderBound.lean`	Uniform positive-order vertical log-product bounds at radius 2^b: $C(b+1)$ for $b \geq 0$, with the legacy Cb corollary for $b \geq 1$.
`NegativeLaplaceVerticalOrdinaryJets.lean`	Exact zero-axis vertical logarithmic jets in terms of ordinary negative-Laplace jets, including the exceptional order-zero normalization.
`PeriodicCorrection.lean`	Continuity of the multiplicatively periodic correction by locally uniform convergence.
`PeriodicMean.lean`	Mean value and normalization of the period-one logarithmic correction.
`PeriodicRegularity.lean`	Differentiability and higher regularity of the periodic correction.
`PeriodicFourier.lean`	Fourier coefficients and nonconstancy of the periodic correction.
`MellinBose.lean`	Mellin transform of the Bose kernel and Gamma/zeta factorization.
`MellinFinitePart.lean`	Finite-part continuation after subtraction of singular Laurent terms.
`BoseFinitePartIntegral.lean`	Convergent integral representation for the regularized Bose/Mellin contribution.
`StieltjesConstant.lean`	Sign-fixed first Stieltjes constant, exact real and complex derivatives at one, and the local zeta Taylor/Laurent interface.
`GammaSecondOrder.lean`	Exact $\Gamma''(1)$, analytic cubic factorization of the quadratic Taylor remainder, and the sharp complex $O(z^3)$ estimate at zero.

Module	Role in the development
`LaplaceCumulantAsymptotics.lean`	Bell-polynomial inversion of normalized Laplace moments through order four and the conditional corrected $O(1/n)$ endpoint comparison.
`LaplacePeriodicSecondOrder.lean`	Second-order transform asymptotics retaining the periodic correction.
Bromwich and Lambert saddle analysis	
`FabiusBromwichInput.lean`	Analytic and decay hypotheses needed to apply Bromwich inversion to the Fabius transform.
`BromwichSaddle.lean`	Generic Fourier–Bromwich inversion and exact change to dimensionless saddle coordinates, without contour shifting.
`FabiusLambertPhase.lean`	Exact lower-Lambert reference coordinate and its domain/inversion identities.
`FabiusLambertRates.lean`	Growth, comparison, and eventual positivity for the exact phase, plus the unconditional reciprocal logarithmic identity and filter-generic Big-O scale conversion.
`FabiusLambertSaddle.lean`	Exact algebraic identities at the lower-Lambert reference radius, without claiming it solves the full residual phase's critical equation.
`FabiusLambertDerivativeBounds.lean`	Uniform bounds for derivatives/cumulants at the saddle scale.
`FabiusLambertMinorArc.lean`	Modulus loss away from the central saddle window.
`FabiusLambertTailFlat.lean`	Exponential tail bounds upgraded to every inverse-power scale.
`FabiusSharpConstant.lean`	Exact normalization constant in the sharp asymptotic formula.
`FabiusSharpLambertMain.lean`	Assembly of the explicit Lambert-phase main term.
`FabiusSharpLambertTransfer.lean`	Transfer of estimates stated in saddle variables to the small-x filter.
`FabiusSharpExactReduction.lean`	Exact decomposition into explicit main term, normalized saddle mass, and forward-tail error.
`FabiusExplicitSharpTransfer.lean`	Conversion between exact sharp expressions and source-facing elementary forms.
`FabiusSharpAsymptoticTransfer.lean`	General transfer lemmas for logarithmic and exponential asymptotic equivalence.
`FabiusSharpAsymptotic.lean`	Correct sharp formula with periodic phase, first error bound, and asymptotic equivalence.
Post-snapshot exponential, cumulant, endpoint-transfer, and interpolation algebra	
`ExponentialPartition.lean`	Weighted partitions and their reciprocal-factorial coefficient sum over arbitrary commutative $\mathbb{Q}$-algebras, including the marked-part and normalized successor recurrences and the field quotient bridge.
`ExponentialBell.lean`	Recurrence-uniqueness bridge identifying the finite weighted-partition family with `SaddleExpansion.expCoeff`.

Module	Role in the development
`MomentCumulantAlgebra.lean`	Factorial normalization, the complete Bell transform and inverse moment–cumulant transform, the classical Bell recurrence, normalized inverse laws, and functoriality over arbitrary commutative $\mathbb{Q}$-algebras.
`EndpointTransferPolynomials.lean`	Universal endpoint-transfer polynomials over $\mathbb{Q}[X]$: recurrence, weighted-partition formula, exact formal exponential series, finite truncation, evaluation naturality, and the coefficients through order three.
`PolynomialExpectationCumulant.lean`	Finite polynomial integrals as coefficient-weighted raw moments and complete Bell transforms of formal cumulants, including the exact zeroth-moment correction and the probability-measure specialization.
`CompleteHomogeneous.lean`	Evaluation and adjoining-variable recurrences for complete homogeneous symmetric polynomials, together with their univariate quotient polynomial, over arbitrary commutative semirings or rings as appropriate.
`GeometricCompleteHomogeneous.lean`	Denominator-free principal specialization $h_n(1, q, \ldots, q^p) = \begin{bmatrix} n+p \\ p \end{bmatrix}_q$, its arbitrary common-scale form over every commutative semiring, and the range-indexed bridge used by interpolation rows.
`LagrangeResidualMoments.lean`	Every higher residual moment of a polynomially exact finite row, with a representation-independent commutative-ring theorem and the distinct-node Lagrange specialization.
`GeometricResidualMoments.lean`	Every positive residual moment on $c, cq, \ldots, cq^p$ over a commutative ring, assuming exactness directly on those scaled nodes, together with distinct-node Lagrange, arbitrary-dilation, and shifted-block field corollaries.
Central integral and all-order machinery	
`FabiusSaddleReduction.lean`	Algebraic reduction of the inversion integral to a normalized central weight.
`FabiusSaddleCentral.lean`	Leading Gaussian approximation in the central window.
`FabiusSaddleCentralLambert.lean`	Central estimate expressed at the exact Lambert saddle.
`FabiusSaddleCentralRadiusAsymptotics.lean`	Growth and compatibility conditions for the chosen central radius.
`FabiusSaddleTail.lean`	Coarse tail bound outside the central window.
`FabiusSaddleReferenceWeight.lean`	Normalized Gaussian reference integrals and identities.
`FabiusSaddleReferenceTail.lean`	Gaussian reference tail estimates.
`FabiusFirstSaddleCorrection.lean`	Explicit first non-Gaussian correction from cubic and quartic cumulants.
`FabiusSaddleCoefficientRecurrence.lean`	Recursive definition and correctness of all-order expansion coefficients.
`FabiusSaddlePolynomialCoefficients.lean`	Polynomial coefficient extraction for the rescaled central exponent.

Module	Role in the development
`FabiusSaddleExpansionCoefficients.lean`	Final scalar coefficients after Gaussian contraction.
`FabiusSaddleExponentAllOrders.lean`	Arbitrary-order expansion of the local exponent with uniform remainder.
`FabiusSaddleCentralAllOrders.lean`	Uniform central-integral expansion to every inverse phase order.
`FabiusSaddleTailAllOrders.lean`	Tail smaller than every requested inverse phase power.
`FabiusSaddleMassAllOrders.lean`	All-order expansion of the normalized saddle mass.
`SaddleExpansionFlat.lean`	Flat-remainder calculus and characterization of equality of full coefficient families modulo $O(\text{scale}^N)$ for every N.
`SaddleExpansionSmallArgument.lean`	Bidirectional transfer of full expansions between the logarithmic coordinate and $x \to 0^+$, for arbitrary normed scalar and vector codomains.
`FabiusSaddleJetClosedForm.lean`	Closed finite formula for periodic saddle jets in derivatives of the periodic correction.
`FabiusSaddleExponentClosedForm.lean`	Closed decomposition of exponent coefficients into a jet term and universal tail.
`FabiusSaddleJetStirling.lean`	Identification of jet weights with shifted signed first-kind Stirling numbers.
`FabiusSaddleLeadingCoefficient.lean`	Exact leading coefficient of the Fabius-specific saddle polynomial.
`FabiusSecondSaddleCorrection.lean`	Closed period-one coefficient at order λ^{-2}.
`FabiusSecondSaddleExpansion.lean`	Public expansion through the second saddle correction with $O(\lambda^{-3})$ remainder.
`GaussianPolynomialContraction.lean`	Linear functional sending even monomials to Gaussian moments and odd monomials to zero.
`GaussianPolynomialWholeIntegral.lean`	Exact whole-line Gaussian integrals of coefficient polynomials.
`GaussianPolynomialTail.lean`	Tail bounds for fixed Gaussian polynomials.
`GaussianPolynomialTailAllOrders.lean`	Uniform tail bounds compatible with arbitrary expansion order.
Lambert formal algebra and final expansion	
`SaddleLogExpansionAlgebra.lean`	Recursive formal logarithm and mutually inverse exponential/logarithmic coefficient transforms, including assumption-free positive-index forms.
`SaddleLogExpansionPowerSeries.lean`	Identification of the recursive logarithm with Mathlib's `PowerSeries.logOf` for unit-constant series.
`FabiusLambertFormalLog.lean`	Formal logarithm/exponential algebra for truncated saddle series.
`FabiusLambertHigherExpansion.lean`	Higher explicit approximants to the exact Lambert phase.
`FabiusLambertAllOrderAlgebra.lean`	Coefficient identities for arbitrary-order Lambert substitution.

Module	Role in the development
`FabiusLambertAllOrderRemainder.lean`	Analytic control of the remainder left by formal Lambert algebra.
`FabiusLambertAllOrderSmallArgument.lean`	Expansion of the exact phase in log(1/x) and log log(1/x) to arbitrary fixed order.
`FabiusFullAsymptoticExpansion.lean`	All-order expansion of log F at the exact phase and weakened elementary-log corollaries.
`FabiusWikipediaMain.lean`	Concise corrected main asymptotic in a public-summary format.
`FabiusWikipediaExpansion.lean`	Corrected higher expansion suitable for comparison with encyclopedic formulas.
`FabiusWikipediaObstruction.lean`	Proof that omission of the nonconstant periodic term invalidates a vanishing residual.
`FabiusQuotientExponentialMismatch.lean`	Rigorous mismatch between Fabius decay and a proposed quotient-of-exponentials fit.
`PaperFabiusAsymptotic.lean`	Aggregator and audit for the coarse, sharp, corrected, and all-order asymptotic program.
Legendre and approximation	
`LegendrePolynomial.lean`	Rodrigues formula, parity, differential equation, monomial coefficients, norms, and endpoint data for Legendre polynomials.
`LegendreSeriesConvergence.lean`	Generic orthogonal-series convergence and coefficient-decay infrastructure for smooth functions.
`FabiusLegendreCoefficients.lean`	Exact even Legendre coefficients expressed through moments and inverse-dyadic values; odd coefficients vanish.
`FabiusLegendreSeries.lean`	Orthogonality, absolute/uniform convergence, and pointwise identification of the up-function series.
`FabiusTranslatedLegendreSeries.lean`	Affine translation to the signed Fabius function and monomial expansion of the translated series.
`FabiusLegendreLeastSquares.lean`	Pythagorean identity and unique least-squares optimality of finite Legendre truncations.
K-fold Thue–Morse audit and auxiliary asymptotics	
`ThueMorseApproximation.lean`	Exact identification of corrected grid samples with moving-center step approximants and pointwise convergence, including the endpoint.
`ThueMorsePrefix.lean`	Iterated prefix recurrences, Prouhet cancellation, exact terminal dyadic zero runs and boundary values, and full signed pre-run reciprocity with its reflected zero-locus equivalence.
`ThueMorseGenerating.lean`	Formal power-series identities, shifted grid recurrence, and exact endpoint refutation of the literal grid and polygonal limits.
`ThueMorseBinomialLog.lean`	Exact integer signed-binomial sum and rational binomial–log formula for the zero–one bit, including the power-of-two log argument and ring-general bridge to the signed convention.
`DraftCounterexamples.lean`	Exact and asymptotic counterexamples to false literal claims in the informal draft.
`StirlingAsymptotics.lean`	Factorial and binomial asymptotics needed for the k-fold scale.

Module	Role in the development
`LowerLambertW.lean`	Correctly defined lower Lambert-type inverse branch, domain, monotonicity, and asymptotics.
`PaperKFoldThueMorse.lean`	Aggregator separating exact results, repaired statements, and formal counterexamples.
Top-level packaging	
`FabiusFunction.lean`	Umbrella import exposing the two paper audits, shape and inverse/non-elementarity theory, computability, negative-Laplace and all-order/closed-form saddle APIs, k-fold audit, and Legendre developments.

Remark D.1. Some helper files are intentionally absent from this table, and a few families may acquire additional leaves as the repository evolves. The umbrella module and the actual import graph remain the definitive inventory. Duplicate-looking module names usually mark distinct proof stages—for example, raw algebra versus scalar specialization, or a central estimate versus its all-order strengthening.

The nine rows headed by `ExponentialPartition.lean`, `ExponentialBell.lean`, `MomentCumulantAlgebra.lean`, `EndpointTransferPolynomials.lean`, and `PolynomialExpectationCumulant.lean`, followed by `CompleteHomogeneous.lean` and `LagrangeResidualMoments.lean`, and finally `GeometricCompleteHomogeneous.lean` and `GeometricResidualMoments.lean`, are a post-snapshot addendum. The first two were checked at code checkpoint `e119db9032ab2641955d47e1da1338a8386aac4d`; the third reflects mainline checkpoint `3989cd9d3`; the endpoint-transfer polynomials were checked at checkpoint `a86aee8e203812816fa5a0e97b9e077a0a6f974e`; and the finite polynomial-integral bridge is present at checkpoint `6e9148873264978eef43181d6bd214c36cdc405a`. The two interpolation rows reflect checkpoints `deefddd335b55bd74b7ac13d782a0b58fb7e2e93` and `cf6cd332660db235675fe3214d5bf757cb424086`. The two geometric specialization rows, including the sixteen exact declarations named in the post-snapshot discussion above, were checked at checkpoint `1e761ed77583e9870ceeaeb2c74ac824094f0f3f`, all on 27 August 2026. The title-page pin and reproduction count continue to describe the older audited snapshot.

E Corrections and critical distinctions

Topic	Informal hazard	Formal treatment
Bounded versus global F	A translation or dyadic formula is written for a clamped function on all of $\mathbb{R}$.	Use `extendedFabius` for the signed global statement; prove agreement with the bounded function on the unit interval.
Original finite convolution	A limiting convolution object is used where a finite-stage identity is required.	State the convolution theorem for each finite row, then pass to the limit through weak convergence.
Step endpoints	Closed interval indicators count a shared endpoint twice.	Define a half-weight endpoint representative when the theorem is pointwise; retain ordinary indicators for almost-everywhere/integral statements.

Topic	Informal hazard	Formal treatment
Fourier equation factor	A transform variable or scaling factor is dropped during differentiation/dilation.	Derive the identity from mathlib's Fourier derivative and affine-map theorems under a fixed convention.
Poisson equations (25) and (32)	Equation (25) omits the translation variable from the exponential, equation (32) retains a factor $1/a$ after the equation has been scaled by a, and the printed specialization carries an unused upper bound $a \leq 1$.	Use the proved normalized identity with $e^{2\pi \mathrm{i} kt/u}$; after multiplying both sides by the spacing, cancel the reciprocal factor. The sharp support specialization needs only $a \geq \frac{1}{2}$.
Positive index	A denominator, derivative order, or truncated product is used at $n = 0$ although the formula assumes $n > 0$.	Add the explicit order hypothesis and prove the zero case separately when meaningful.
Removable singularity	A quotient such as $(e^z - 1)/z$ is treated as globally defined.	Define a total function with the limiting value at zero and prove a global power-series theorem.
Dyadic representation	A formula is proved only for a reduced numerator even though later sums produce unreduced fractions.	Prove refinement invariance and formulate global correctness for arbitrary representatives.
Taylor series	Exact finite Taylor truncation is mistaken for local analyticity.	Use a finite-order Taylor theorem with vanishing remainder; separately prove the analytic-at predicate fails.
Logarithmic delay equation	A branch-dependent derivative law is differentiated without a domain.	State the transformed identity only for $t \geq 1$ and use eventual filters thereafter.
Periodic asymptotic term	A bounded log-periodic correction is absorbed into an $o(1)$ or inverse-log error.	Prove the correction is nonconstant and exhibit phase subsequences with different residual limits.
Binary-reduction series	The outer sum begins after the endpoint contribution.	Start at $m = 0$ or include the missing boundary term explicitly so finite telescoping is exact.
Finite q dependence	Independence of the limiting value is promoted to independence of every approximant.	Preserve the complex q parameter at finite depth; prove its effect tends to zero through complex-shift estimates.
Theorem 20 naturality	A proof-internal sentence asserts that $d_k N_n$ is natural.	The corrected integral quantity is $2 d_k N_n$; `two_mul_halfMoment_mul_reshetnikovMultiplier_isNatural` and its oddness strengthening retain the missing factor 2.

Topic	Informal hazard	Formal treatment
Saddle expansion composition	A truncated Lambert expansion is substituted to all orders without a composition remainder proof.	State the primary all-order theorem at the exact phase and prove elementary-log expansions separately with implicit-function error bounds.
Quotient exponential fit	Numerical agreement over a finite range is treated as asymptotic equivalence.	Compare logarithmic scales and prove the proposed quotient has the wrong endpoint decay.
K-fold normalization	A draft normalization or global error is assumed from plots or low-depth data.	Evaluate exact small cases and formalize counterexamples before proving a repaired statement.
K-fold polygon	Convergence of corrected grid samples is restated as convergence of the literal interpolating polygon.	The literal polygon is refuted at $x = 1$; only the normalized grid samples are identified with moving-center step approximants and proved convergent.
K-fold local error	The proposed $4h$ error bound is inferred from a picture.	At $k = 2, j = 1$, the proposed slope is -2 while $F'(1/4) = 1$, giving error $3 > 4h = 1$.
K-fold equation (7)	An unbounded proxy sequence is assigned a finite maximum.	For every $x > 0$, its successive-ratio analysis gives growth like $x2^{n+1}/(n + 1)$, so no displayed maximum exists.
K-fold equation (10)	A linear residual is hidden inside $O(\log n)$.	Substituting the printed stationary equation leaves an explicit $+n$ term, which cannot be absorbed into a logarithmic error.

E.1 What these corrections teach

The corrections fall into four recurring classes. Domain errors arise when a local functional equation is used globally. Representation errors arise when a bounded object is substituted for a signed extension or an almost-everywhere representative is substituted in a pointwise claim. Normalization errors arise from Fourier, convolution, and dyadic scaling conventions. Asymptotic errors arise when a bounded oscillation is mistaken for a small remainder or when a formal series is composed without analytic control.

Lean detects each class differently. A domain error appears as an unprovable interval side condition. A representation error appears as a concrete endpoint contradiction. A normalization error leaves an unmatched scalar factor. An asymptotic error usually survives local algebra but fails at the limit theorem; the repository then proves a negative result by constructing a subsequence or comparing scales.

F Lean and mathematical glossary

Term or identifier	Meaning in this development
`UnitInterval`	The subtype $\{x : \mathbb{R} \mid 0 \le x \le 1\}$, used as the codomain of the bounded function.
`BoundedFabius`	A real-input function whose values carry the unit-interval bound in their type.
`fabiusReal`	Real-valued projection of a subtype-valued bounded function, suitable for calculus and integration.
`IsFabius`	Abstract specification combining tail values, smoothness, symmetry, and the left-half derivative law.
`fabius` / `fabius_spec`	The canonical constructed object and the theorem that it satisfies the specification.
`rvachevUp`	Compactly supported even twin used in Fourier and probability arguments.
`extendedFabius`	Signed globally defined extension assembled from Thue–Morse-weighted translates of the up-function.
`HasDerivAt`	Pointwise derivative proposition carrying differentiability and the derivative value together.
`ContDiff`	Smoothness predicate to finite or infinite order.
`tsum`	Infinite sum over a type; translate sums are proved locally finite before this is manipulated.
`HasSum`	Proposition that a series has a specified sum; often more convenient than rewriting a `tsum`.
`Finset.rangen`	Finite set $\{0, \ldots, n-1\}$, the dominant indexing object for recurrences and finite products.
`Nat.strong_induction_on`	Well-founded induction giving hypotheses for every smaller natural index.
`IsBigO`, `IsLittleO`	Filter-based asymptotic bounds and negligibility.
`atTop`	Filter describing a variable tending to positive infinity.
`nhdsWithin0(Set.Ioi0)`	Right-hand approach to zero, the endpoint filter used for $x \downarrow 0$.
`eventually`	A property holding on some set belonging to a filter; used to encode large-variable thresholds.
`Summable`	Absolute or unconditional summability in the relevant normed additive group.
`intervalIntegral`	Oriented integral over real endpoints, useful for FTC and integration by parts.
`MeasureTheory.Integrable`	Whole-space integrability; usually proved from compact support and continuity.
`SchwartzMap`	Smooth rapidly decreasing function type used for Fourier injectivity and Poisson summation.
`fabiusDyadicValue`	Executable exact rational evaluator for a dyadic input representation.
`fabiusDyadicUnitAux`	Well-founded highest-bit recursion at the core of unit-interval evaluation.
`halfMoment` / `moment`	Exact rational recurrence sequences later identified with analytic integrals.
`halfMomentNumerator` / `momentNumerator`	Division-free natural recurrences used for parity and denominator proofs.
`binaryWeight`	Number of one-bits of a natural number.
`thueMorseSign`	Sign $(-1)^{s_2(n)}$.
`halfQBinomial`	Exact q-binomial coefficient specialized to $q = 1/2$.

Term or identifier	Meaning in this development
`field_simp`	Tactic for clearing field denominators after nonvanishing hypotheses have been supplied.
`norm_cast` / `push_cast`	Tactics normalizing canonical casts across algebraic operations.
`ring_nf`	Polynomial normalizer used after divisions, casts, and finite sums are brought to algebraic normal form.
`filter_upwards`	Tactic for collecting eventual hypotheses and proving a pointwise eventual conclusion.

G Example exploration snippets

The following snippets are intentionally schematic where theorem argument order may change. They show the style of interaction rather than replacing editor inspection of the current declarations.

G.1 Inspect the public objects

```
import FabiusFunction

open Fabius

#check UnitInterval
#check BoundedFabius
#check IsFabius
#check fabius
#check fabius_spec
#check rvachevUp
#check extendedFabius
```

G.2 Evaluate exact dyadic data

```
import FabiusFunction.Arithmetic

open Fabius

#eval fabiusInversePowTwoTable 8
#eval fabiusDyadicValue 1 1
#eval fabiusDyadicValue 5 4
#eval extendedFabiusDyadicValue 13 5
```

The returned objects are exact rationals. To evaluate a rational input through the recognition layer, inspect the declarations near `IsDyadicRational`, `dyadicExponent?`, and the evaluator wrappers in `Arithmetic.lean`.

G.3 Work generically under the specification

```
import FabiusFunction.Differential

open Fabius
```

```
variable (F : BoundedFabius) (hF : IsFabius F)

#check hF.symmetry_all
-- Search within Differential.lean for the iterated derivative
-- theorem whose interval hypotheses fit the desired dyadic cell.
```

The generic style is useful for proving a new consequence once and applying it both to the canonical function and to any future equivalent construction.

G.4 Find source-facing arithmetic theorems

```
import FabiusFunction.Paper06487

open Fabius

#check proposition_one
#check theorem_seven
#check theorem_twenty
#check theorem_twenty_one
#check conjecture_sixteen
```

A declaration representing a conjecture is a definition or proposition describing the assertion, not a proof of it.

G.5 Inspect asymptotic interfaces

```
import FabiusFunction.PaperFabiusAsymptotic

open Fabius

-- Use #check on the named main, obstruction, and full-expansion
-- theorems, then follow their exact-phase helper declarations.
#check Fabius.log_fabius_sub_correctedWikipediaMain_isBigO
#check Fabius.log_fabius_sub_WikipediaElementaryMain_not_isBigO
#check Fabius.log_fabius_dyadicReal_sub_sharpLambertExpansion_isBigO
```

Because filter statements can be long, editor hover information is often more useful than printing the entire type in a terminal.

G.6 Search tactics

Within an editor, useful text searches include:

```
IsFabius -> HasDerivAt
fabiusDyadicValue -> Real
rvachevUp -> Fourier
negativeLaplaceLog
HasAsymptoticExpansion
Legendre -> HasSum
```

Searching by the transition one wants—rather than by a source theorem number—usually finds the strong internal lemma.

H Reproduction and trust checks

H.1 Pin the inspected tree

This edition was checked against the full commit printed on the title page, not against an unspecified moving branch. A reproduction run should begin with:

```
git rev-parse HEAD
source_pin=eaea1b9afe2b7ff0b7dd880bd1710e303dec6d80
test "$(git rev-parse "$source_pin^{commit}")" = "$source_pin"
git diff --exit-code "$source_pin" -- \
  Analysis/FabiusFunction/Lean
git show -s --format=%cs "$source_pin"
lake env lean --version
rg --files Analysis/FabiusFunction/Lean/FabiusFunction \
  -g '*.lean' | wc -l
```

At the pinned snapshot the last command reports 189 leaf modules. A different count is not automatically an error, but it means this guide's module inventory must be compared with the newer umbrella import. Do not run `lake update`: the committed manifest is part of the audited environment.

H.2 Build and inspect trust

The release-level library check is

```
LAKE_JOBS=1 LEAN_NUM_THREADS=0 lake build +FabiusFunction
```

The repository policy requires one prefixed module target per Lake invocation, with `LAKE_JOBS=1` and `LEAN_NUM_THREADS=0`, including during debugging. It also requires a source audit for declarations that bypass ordinary proof checking:

```
rg -n '\b(sorry|admit)\b' \
  Analysis/FabiusFunction/Lean -g '*.lean'
rg -n '\b(axiom|opaque)\b' \
  Analysis/FabiusFunction/Lean -g '*.lean'
```

The first search is empty at the pinned snapshot. The second intentionally shows two comment-only uses of the word *opaque*; inspect the raw matches to confirm that neither is a declaration. For representative public theorems, a small scratch file may additionally request:

```
import FabiusFunction

#print axioms Fabius.fabiusDyadicValue_cast
#print axioms Fabius.log_fabius_sub_correctedWikipediaMain_isBigO
```

Any reported axioms must be drawn from the repository's permitted set `propext`, `Classical.choice`, and `Quot.sound`; an individual theorem may use only a subset.

H.3 Rebuild this document

From this document's directory, use exactly three passes:

```
pdflatex -interaction=nonstopmode -halt-on-error \
  fabius_lean_walkthrough.tex
pdflatex -interaction=nonstopmode -halt-on-error \
  fabius_lean_walkthrough.tex
pdflatex -interaction=nonstopmode -halt-on-error \
  fabius_lean_walkthrough.tex
rm -f fabius_lean_walkthrough.aux \
  fabius_lean_walkthrough.log \
  fabius_lean_walkthrough.out \
  fabius_lean_walkthrough.toc
```

The source and generated PDF belong in the same commit. A successful TeX build checks syntax and references, but it does not validate mathematical prose; that is why this walkthrough also cites exact declaration names and records its nonclaims.

Bibliographic and repository notes

[1] Vladimir Reshetnikov, *ProveIt: Analysis/FabiusFunction*, GitHub repository, source tree and README inspected 25 August 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/eaea1b9afe2b7ff0b7dd880bd1710e303dec6d80/Analysis/FabiusFunction

[2] Juan Arias de Reyna, *An infinitely differentiable function with compact support: definition and properties*, arXiv:1702.05442 (2017). https://arxiv.org/abs/1702.05442

[3] Juan Arias de Reyna, *Arithmetic of the Fabius function*, arXiv:1702.06487, version 3 (2017). https://arxiv.org/abs/1702.06487

[4] Leonardo de Moura and Sebastian Ullrich, *The Lean 4 Theorem Prover and Programming Language*, Automated Deduction – CADE 28, 2021.

[5] The mathlib community, *The Lean mathematical library*, CPP 2020 and continuously maintained Lean 4 library. https://github.com/leanprover-community/mathlib4

Document metadata

Title	The Fabius Function in Lean: A Guided Walkthrough of the ProveIt Formalization
Source snapshot	commit `eaea1b9afe2b7ff0b7dd880bd1710e303dec6d80` on `codex/fabius-both-papers`, inspected 25 August 2026
Toolchain recorded by source	Lean 4.32.0; repository-pinned mathlib revision
Primary path	`Analysis/FabiusFunction/Lean/`
Umbrella module	`FabiusFunction.lean`
Document purpose	Human-readable structural and proof-oriented guide to the Lean development